Flexible Services and Manufacturing Journal (2022) 34:156–203
https://doi.org/10.1007/s10696-021-09403-0

Integrating preventive maintenance to two-stage assembly flow shop scheduling: MILP model, constructive heuristics and meta-heuristics

Zikai Zhang[1,2] · **Qiuhua Tang**[1,2]

Accepted: 30 January 2021 / Published online: 3 March 2021

Abstract
In this paper, flexible preventive maintenance (PM) activities are incorporated into two-stage assembly flow shop scheduling where m dedicated machines in the fabrication stage and one machine in the assembly stage. The operational status of each machine is described by a continuous variable, maintenance level. The maintenance level value is inversely proportional to the processing time. Once a PM activity is performed, this value will return to the initial value. Different from the PM at fixed predefined time intervals, flexible PM can be carried out at any time point, but the maintenance levels are not less than 0. Hence, a MILP model with maintenance level constraints is formulated to minimize the total completion time and maintenance time. Regarding the methods, a latest PM decision strategy is proposed to determine the execution time of PM activities. This new strategy is embedded into 15 constructive heuristics and 7 meta-heuristics (three variants of iterated local search, three variants of Q-learning-based ant colony system with local search and a Q-learning-based hyper-heuristics) to address this new problem. The final experimental analysis demonstrates the significance of the integrated model and the effectiveness of the proposed constructive heuristics and meta-heuristics.

Keywords Assembly flow shop · Preventive maintenance · Constructive heuristic · Meta-heuristic · Q-learning

✉ Qiuhua Tang
tangqiuhua@wust.edu.cn

Zikai Zhang
zhangzikai0703@gmail.com

[1] Key Laboratory of Metallurgical Equipment and Control Technology of Ministry of Education, Wuhan University of Science and Technology, Wuhan, China

[2] Hubei Key Laboratory of Mechanical Transmission and Manufacturing Engineering, Wuhan University of Science and Technology, Wuhan, China

1 Introduction

In the current manufacturing environment, competitive market and customization requirements force enterprises to organize flexible production modes to produce a variety of products in large volume (Komaki et al. 2019). Accordingly, an assembly flow shop (AF) is developed to meet the above production. AF mainly contains two stages: fabrication and assembly. A variety of products are produced by firstly processing all their components at the fabrication stage and then jointly assembling components at the assembly stage. The corresponding scheduling problem is intended to find an optimal product sequence subject to specific objectives. Nowadays, due to its increasing industrial and service applications, the assembly flow shop scheduling problem has received the attention of more and more researchers. To our knowledge, the typical and successful applications of assembly flow shop include fire engine (Lee et al. 1993), computers (Potts et al. 1995), plastic products (Allahverdi and Aydilek 2013), clothes (Yokoyama 2004), automobiles (Fattahi et al. 2013; Liao et al. 2015), distributed database systems (Allahverdi and Al-Anzi 2009) and multi-page invoice printing systems (Zhang et al. 2010).

In this paper, we concentrate on a two-stage assembly flow shop scheduling problem where the fabrication stage contains several dedicated machines to respectively process corresponding components and the assembly stage just has one machine to assemble these components into a qualified product. So far, the two-stage assembly flow shop scheduling problem has been extensively investigated in the literature. According to the objective functions, these known researches can be classified into five categories (Komaki et al. 2019): (1) minimizing the maximum completion time; (2) minimizing the total completion time; (3) minimizing maximum tardiness or total tardiness; (4) minimizing maximum lateness or total lateness; (5) other objectives such as minimizing costs and minimizing orders lead time. One of the objectives of this paper is to minimize the total completion time. In addition, regarding constraints, parallel machines, buffer, setup times and release time are commonly discussed in two-stage assembly flow shop scheduling problems.

Although more attention is given to two-stage assembly flow shop scheduling problems, there are still many limitations in current literature with respect to most industrial reality. Contemporary literature is based on the hypothesis that machines are always available and never break. However, in the actual production, machines are inevitably subject to some unavailable periods due to unexpected failure or preventive maintenance with time elapsing. This situation will result in production halts and even bring huge production and economic loss to the enterprises. Hence, the joint of scheduling and preventive maintenance needs to reach the improvement of productivity and reliability simultaneously. In this paper, we embed PM operations into production scheduling plans, and thus we need to determine the product sequence along with PM operation execution time points. To our knowledge, the integration of two-stage assembly flow shop scheduling and the PM has rarely been addressed before in the scientific literature.

This paper principally contains the following contributions:

- A MILP model with maintenance level is developed to minimize the total completion time and total maintenance time of the integration of two-stage assembly flow shop scheduling and PM.
- A latest PM decision strategy is proposed to determine the execution time of PM activities, and this new strategy is embedded into 15 constructive heuristics to solve this new problem fast and efficiently.
- 7 meta-heuristics, including three variants of iterated local search (ILS), three variants of Q-learning-based ant colony system with local search (QACS_LS) and a Q-learning-based hyper-heuristics (QHH), are developed to further improve the quality of solutions. For the proposed ILS, we design three local search procedures to enhance the exploration ability and two perturbation procedures to avoid falling into local optima. For QACS_LS, we embed the local search procedures in the ant colony system, but also use Q-learning to determine an appropriate parameter combination in each iteration. For QHH, we design Q-learning-based selection at a high level and 15 neighbor operators at a low level.

The remainder of this paper is organized as follows: A review of the related literature is given in Sect. 2. A new MILP model is formulated to descript the integration of two-stage assembly flow shop scheduling and PM in Sect. 3. Then, 15 constructive heuristics are improved to tackle this new problem in Sects. 4 and 7 meta-heuristics are designed in Sect. 5. Experimental results are reported in Sect. 6. Finally, conclusions and future work are discussed in Sect. 7.

2 Literature review

Since this paper aims to study the integration of two-stage assembly flow shop scheduling and PM, this section first investigates the studies on two-stage assembly flow shop scheduling, and then reports PM considered in scheduling problems.

2.1 Current research status on two-stage assembly flow shop scheduling

As noted above, two-stage assembly flow shop scheduling can be divided into five categories according to objective functions. Among studies on makespan, Lee et al. (1993) introduced the concept of two-stage assembly flow shop scheduling and proved that it is strong NP-hard. Then, they designed a branch and bound solution scheme along with three heuristics. Since then, Fattahi et al. (2013) assumed that the fabrication stage is a hybrid flow shop to produce all components of products, and then designed a series of heuristic algorithms to minimize the makespan. Navaei et al. (2013) investigated two-stage assembly flow shop scheduling where multiple non-identical assembly machines were involved in the second stage with the minimization of makespan. Komaki et al. (2015) studied two-stage assembly flow

shop scheduling where the fabrication stage is a hybrid flow shop layout. And they further developed two meta-heuristic techniques based on an artificial immune system to minimize the makespan. Liao et al. (2015) studied batch setup times in two-stage assembly flow shop scheduling and developed an efficient heuristic based on several properties to minimize makespan. Jung et al. (2017) considered dynamic component-sizes and setup times into two-stage assembly flow shop scheduling. To solve this problem, they further designed three genetic algorithms with different chromosome representations. Lin (2018) studied two-stage three-machine assembly flow shop scheduling with a learning effect, and developed a cloud theory-based simulated annealing algorithm and an iterated greedy algorithm to find near-optimal solutions with the minimization of makespan. Wu et al. (2018a) considered position-based learning effect into two-stage assembly flow shop scheduling where two machines were involved in fabrication stage and one machine was involved in assembly stage, and proposed three heuristics along with three versions of simulated annealing algorithm.

Regarding the total completion time objective, Earlier studies can refer to these several literature (Yokoyama and Santos 2005; Tozkapan et al. 2003; Yokoyama 2001). Recently, Sung and Kim (2008) addressed the two-stage multi-machine assembly flow shop scheduling and developed a branch and bound algorithm to solve it. Allahverdi and Al-Anzi (Allahverdi and Al-Anzi 2009) proposed three heuristics to minimize the total completion time of two-stage assembly flow shop scheduling with setup times. Al-Anzi and Allahverdi (2013) further studied two-stage assembly flow shop scheduling where the assembly stage consisted of two independent and identical assembly machines, and solved it by an artificial immune system heuristic. Framinan and Perez-Gonzalez (2017) developed an efficient constructive heuristic and a variable local search algorithm to minimize the total completion time. The final experimental results indicated that the proposed heuristic and algorithm respectively outperformed the existing heuristics and the best existing meta-heuristics. Lee (2018) proposed a branch-and-bound algorithm with six lower bounds and four constructive heuristics to generate near-optimal solutions. Wu et al. (2018b) studied two-stage three-machine assembly flow shop scheduling with learning phenomenon, and designed six versions of hybrid particle swarm optimization (PSO) algorithm to minimize the total completion time. Wu et al. (2019) integrated three machines, two agents and learning phenomenon into two-stage assembly flow shop scheduling. To achieve the optimization of total completion time, they promoted a branch-and-bound algorithm and four versions of hybrid particle swarm optimization algorithms respectively for small-size and big-size jobs. Talens et al. (2020) considered two-stage assembly flow shop scheduling with multi-machine in the assembly stage to minimize the total completion time. They first designed a fast constructive heuristic embedded with some specific knowledge of the problem domain, and then proposed a beam search based on the above constructive heuristic.

With respect to the maximum tardiness or total tardiness objective, Allahverdi and Aydilek (2013) proposed an insertion algorithm, a genetic algorithm, two versions of simulated annealing algorithm and two versions of cloud theory-based simulated annealing algorithm to reduce the total tardiness. Allahverdi et al. (2016) investigated the two-stage assembly flow shop scheduling with setup times and

proposed two new algorithms to minimize the total tardiness. Aydilek et al. (2017) considered setup times into two-stage assembly flow shop scheduling and proposed a new hybrid simulated annealing and insertion algorithm to minimize the maximum tardiness. Kazemi et al. (2017) investigated two-stage assembly flow shop scheduling with a batched delivery system, and developed an imperialist competitive algorithm and a hybrid algorithm incorporated with the dominance relations. Chung and Chen (2019) considered distinct due windows in two-stage assembly flow shop scheduling, and proposed a complete immunoglobulin-based artificial immune system algorithm to optimize the sum of weighted earliness and tardiness.

As for the maximum lateness or total lateness objective, Al-Anzi and Allahverdi (2007) developed a self-adaptive differential evolutionary heuristic to minimize maximum lateness of two-stage assembly scheduling problem with setup times. Furthermore, studies have likewise been investigated on other objectives such as costs and orders lead time. Navaei et al. (2014) addressed two-stage assembly flow shop scheduling with non-identical assembly machines at the second stage and setup times to minimize a sum of holding and delay costs. Blocher and Chhajed (2007) aimed to minimize customer order lead time of the two-stage assembly supply chain.

2.2 PM in scheduling problems

It can be observed from the above literature that the current researches mainly focus on several specific constraints in two-stage assembly flow shop scheduling and then aim to develop efficient constructive heuristics or meta-heuristics. Nevertheless, they ignore the impact of PM on the production schedule, which is emphasized in the above introduction. To our best knowledge, only two papers investigate PM in two-stage assembly flow shop scheduling. Seidgar et al. (2016a) considered PM policies into two-stage assembly flow shop scheduling, and proposed imperialist competitive algorithm and genetic algorithm to minimize the makespan. Then based on the above research, Seidgar et al. (2016b) further studied bi-objective optimization for the integration of two-stage assembly flow shop scheduling. They integrated PM into scheduling through maintenance intervals. Besides, more literature can refer to the joint of PM and other production scheduling such as flow shop and job shop. Generally, current PM approaches considered in scheduling fall into two categories: time-based maintenance and condition-based maintenance. The former is developed based on historical failure data; while the latter is based on current conditions and real-time data. Regarding time-based maintenance, it is usually integrated into flow shop scheduling (Ye et al. 2020; Mosheiov et al. 2018; Wang and Liu 2014; Ruiz et al. 2007; Naderi et al. 2011; Seif et al. 2019), job shop scheduling (Khoukhi et al. 2017; Li et al. 2014; Zhou et al. 2012; Li and Pan 2012), open shop scheduling (Mosheiov et al. 2018; Sheikhalishahi et al. 2019). With respect to condition-based maintenance, Safari et al. (2009) proposed a hybrid simulated annealing-tabu search to tackle m-machine flow shop scheduling with condition-based maintenance constraint. Bock et al. (2011) considered flexible PM in single-machine scheduling where PM activities were performed according to the remaining useful life of the machine. Yu and Seif (2016) and Miyata et al. (2019) respectively

extended maintenance level to m-machine flow shop and no-wait flow shop scheduling. Zandieh et al. (2017) improved the imperialist competitive algorithm to tackle flexible job shop scheduling under condition-based maintenance. Shamsaei and Van Vyve (2016) integrated the condition-based maintenance scheduling into production planning.

Inspired by the above condition-based maintenance, this paper attempts to extend maintenance level into two-stage assembly flow shop scheduling. Each machine has an initial maintenance level identified by the optimal maintenance interval (this interval is calculated using the Weibull distribution). The maintenance level value is changed in two cases: (1) when a product is processed on a machine, the corresponding maintenance level will decrease; (2) PM activity can restore the maintenance level to the initial value. Once the value of the maintenance level goes down to 0, there is a high probability of machine failure. Hence, it is necessary to decide the product sequence and PM operation execution time point to ensure efficient production and avoid machine breakdown simultaneously.

3 Problem formulation

Before formulating this new problem, Table 1 presents the notation related to our paper. According to the classification in Framinan et al. (2019), our new problem can be denoted as $DP_m \rightarrow 1|PM| \sum_{i=1}^{n} C_{i,m+1} + \sum_{i=1}^{n} \sum_{k=1}^{m+1} Y_{ik} \cdot dpm_k$. In this problem, n products (each one consists of m components) need to be processed in the fabrication

Table 1 Indices, parameters and variables of the proposed model

Indices:	
j	Index of product, $j = 1, \ldots, n$;
i	Index of position in the product sequence, $i = 1, \ldots, n$;
k	Index of machine, $k = 1, \ldots, m+1$;
Parameters:	
n	The number of products;
m	The number of machines at the first stage;
p_j	The processing time of product j at the assembly stage;
t_{jk}	The processing time of product j on k-th machine (k-th parts of product j) at the first stage;
ml_k^0	The initial maintenance level of machine k;
μ_k	Deterioration coefficient of machine k;
dpm_k	Maintenance duration of machine k;
M	A number sufficiently large;
Variables:	
C_{ik}	Continuous variable. Complete time of the product in position i on the k-th machine;
ML_{ik}^a	Continuous variable. The maintenance level of machine k after processing the product in position i;
X_{ji}	Binary variable. 1: if product j is assigned in position i; 0: otherwise
Y_{ik}	Binary variable. 1: if PM activity is performed immediately before the start of the product in position i on machine k; 0: otherwise

and assembly stages. Clearly, m components are produced on m dedicated parallel machines respectively at the fabrication stage, and then assembled into one product on one assembly machine at the assembly stage. Each machine has an initial maintenance level ml_k^0. When the machine is on work, its maintenance level goes down linearly and the deterioration coefficient is denoted as μ_k. However, maintenance levels are not allowed to reach values below zero, otherwise, PM activities are carried out to restore the levels to initial maintenance levels. The objective functions are to minimize the total completion time $\sum_{i=1}^{n} C_{i,m+1}$ and the total maintenance time $\sum_{i=1}^{n} \sum_{k=1}^{m+1} Y_{ik} \cdot dpm_k$. The corresponding hypotheses of this problem are as follows:

1. Maintenance duration is different according to the machine;
2. Random failures are not considered;
3. Maintenance levels are not allowed to reach values below zero;
4. Setup times are not considered;
5. the buffers between fabrication and assembly stages are ignored.

3.1 Mathematical model

Objective function:

$$min \sum_{i=1}^{n} C_{i,m+1} + \sum_{i=1}^{n} \sum_{k=1}^{m+1} Y_{ik} \cdot dpm_k \tag{1}$$

Subject to:

$$\sum_{j=1}^{n} X_{ji} = 1, \forall i = 1, \ldots, n \tag{2}$$

$$\sum_{i=1}^{n} X_{ji} = 1, \forall j = 1, \ldots, n \tag{3}$$

$$C_{0k} = 0, \forall k = 1, \ldots, m+1 \tag{4}$$

$$C_{ik} \geq C_{i-1,k} + \sum_{j=1}^{n} t_{jk} \cdot X_{ji} + Y_{ik} \cdot dpm_k, \forall i = 1, \ldots, n, k = 1, \ldots, m \tag{5}$$

$$C_{i,m+1} \geq C_{i-1,m+1} + \sum_{j=1}^{n} p_j \cdot X_{ji} + Y_{i,m+1} \cdot dpm_{m+1}, \ \forall i = 1, \ldots, n \tag{6}$$

$$C_{i,m+1} \geq C_{ik} + \sum_{j=1}^{n} p_j \cdot X_{ji}, \forall\, i = 1, \ldots, n, k = 1, \ldots, m \tag{7}$$

$$ML_{0k}^{a} = ml_{k}^{0}, \forall k = 1, \ldots, m+1 \tag{8}$$

$$ML_{ik}^{a} - \left(ml_{k}^{0} - \mu_k \cdot \sum_{j=1}^{n} t_{jk} \cdot X_{ji} \right) \leq M \cdot \left(1 - Y_{ik}\right), \forall\, i = 1, \ldots, n,\ k = 1, \ldots, m \tag{9}$$

$$ML_{ik}^{a} - \left(ml_{k}^{0} - \mu_k \cdot \sum_{j=1}^{n} t_{jk} \cdot X_{ji} \right) \geq -M \cdot \left(1 - Y_{ik}\right), \forall\, i = 1, \ldots, n,\ k = 1, \ldots, m \tag{10}$$

$$ML_{ik}^{a} - \left(ML_{i-1,k}^{a} - \mu_k \cdot \sum_{j=1}^{n} t_{jk} \cdot X_{ji} \right) \leq M \cdot Y_{ik}, \forall\, i = 1, \ldots, n,\ k = 1, \ldots, m \tag{11}$$

$$ML_{ik}^{a} - \left(ML_{i-1,k}^{a} - \mu_k \cdot \sum_{j=1}^{n} t_{jk} \cdot X_{ji} \right) \geq -M \cdot Y_{ik}, \forall\, i = 1, \ldots, n,\ k = 1, \ldots, m \tag{12}$$

$$ML_{i,m+1}^{a} - \left(ml_{m+1}^{0} - \mu_{m+1} \cdot \sum_{j=1}^{n} p_j \cdot X_{ji} \right) \leq M \cdot \left(1 - Y_{i,m+1}\right),\ \forall\, i = 1, \ldots, n \tag{13}$$

$$ML_{i,m+1}^{a} - \left(ml_{m+1}^{0} - \mu_{m+1} \cdot \sum_{j=1}^{n} p_j \cdot X_{ji} \right) \geq -M \cdot \left(1 - Y_{i,m+1}\right), \forall\, i = 1, \ldots, n \tag{14}$$

$$ML_{i,m+1}^{a} - \left(ML_{i-1,m+1}^{a} - \mu_{m+1} \cdot \sum_{j=1}^{n} p_j \cdot X_{ji} \right) \leq M \cdot Y_{i,m+1}, \forall\, i = 1, \ldots, n \tag{15}$$

$$ML_{i,m+1}^{a} - \left(ML_{i-1,m+1}^{a} - \mu_{m+1} \cdot \sum_{j=1}^{n} p_j \cdot X_{ji} \right) \geq -M \cdot Y_{i,m+1},\ \forall\, i = 1, \ldots, n \tag{16}$$

$$C_{ik} \geq 0, \forall i = 1, \ldots, n, k = 1, \ldots, m+1 \tag{17}$$

$$ML_{ik}^{a} \geq 0, \forall i = 1, \ldots, n, k = 1, \ldots, m+1 \tag{18}$$

$$X_{ji} = \{0, 1\}, \forall j = 1, \ldots, n, i = 1, \ldots, n \tag{19}$$

$$Y_{ik} = \{0, 1\}, \forall i = 1, \ldots, n, k = 1, \ldots, m + 1 \tag{20}$$

The objective function (1) is intended to minimize the total completion time and total maintenance time. the first term of the objective is to minimize the total completion time and the second one relates to the total maintenance time. Constraint (2) ensures that each position just has one product. Constraint (3) implies that each product is assigned to one position. Constraint (4) indicates that all machines are available at the beginning. Constraint (5) implies that in the fabrication stage, each product begins to be processed only when the products in its previous positions have finished. And if a PM activity is carried out immediately before this product, the start time needs to be pushed back maintenance duration. Constraints (6–7) limit the start time of each product in the assembly stage. Specifically, constraint (6) is similar to the constraint (5); Constraint (7) ensures that the product begins to be assembled only when all its components have been processed. Constraint (8) defines that all machines are at the initial maintenance levels at the beginning. Constraints (9–12) limit the maintenance levels of fabrication machines. Specifically, if a PM activity is performed immediately before the start of the product j in position i on machine k, Constraints (9–10) become active and restore the maintenance level of machine k to the initial maintenance level ml_k^0 before processing the product j; otherwise, constraints (11–12) become active and imply that the maintenance level decreases linearly. Constraints (13–16) are similar to constraints (9–12) but aim to limit the maintenance levels of assembly machines. Constraint (17) requires that all the completion times are greater than 0. Constraint (18) addresses that all the maintenance levels are not allowed to reach values below 0. Constraints (19–20) indicate binary variables.

3.2 Determination of initial maintenance levels and deterioration coefficient

The determination of PM activities is based on the statistical theory of reliability. Generally, the lifetime of machines often follows a Weibull probability distribution, $T \sim W[\theta, \beta]$, due to its flexibility to characterize the time to failure of machines with non-constant failure rates (Ruiz et al. 2007). Then, the reliability function, instantaneous failure rate, the mean time to failure ($MTTF$) and availability are identified according to Eqs. (21–23). Among them, β and θ are shape parameter and scale parameter respectively.

$$R(t) = exp\left[-\left(\frac{t}{\theta}\right)^{\beta}\right], \ t > 0 \tag{21}$$

$$h(t) = \frac{\beta}{\theta}\left(\frac{t}{\theta}\right)^{\beta-1}, \ t > 0 \tag{22}$$

$$MTTF = \theta \cdot \Gamma\left[1 + \frac{1}{\beta}\right] \tag{23}$$

Based on the above theoretical basis, one of the popular PM policies identifies an optimal PM interval by maximizing the availability given by Eq. (24) (Ruiz et al. 2007). Among them, t_r is the number of time units the repair takes and t_p is the number of time units of the PM. It is suggested that PM activity is carried out at T_{PMop} interval. Hence, in this paper, we use the optimal maintenance interval to represent the initial maintenance level, and further the deterioration coefficient is set 1.0. In this policy, it can be ensured that at T_{PMop} interval the machine can be restored as -good-as-new condition and its reliability is also maintained at an acceptable level.

$$T_{PMop} = \theta \cdot \left[\frac{t_p}{t_r(\beta - 1)}\right]^{1/\beta} \tag{24}$$

3.3 An illustrative example

An illustrative example with 20 products and 3 machines (2 fabrication machines and 1 assembly machine) is presented to give a better insight of this new problem. The processing and assembly times are shown in Table 2. It is assumed that the shape parameters of 3 machines are set as 3, 3 and 4, respectively, and the scale parameters are all equal to 1500. $t_r = 8$ and $t_p = 1$. The maintenance durations on 3 machines are respectively 84, 13 and 89. According to Eq. (24), the initial maintenance levels of 3 machines are 595.28, 595.28 and 677.70, respectively. All the deterioration coefficients are set 1.0 as mentioned above. This example is solved through the proposed mathematical model, coded by the IBM Cplex. The final optimal solution is given in Fig. 1.

In this figure, the final product sequence is {1, 6, 13, 2, 15, 20, 17, 5, 9, 14, 3, 18, 16, 19, 10, 7, 11, 8, 4, 12}, and PMs are carried out on machines 1–3 before the processing of product 16. Note that it is a coincidence that PMs on three machines are behind the same product. The total completion time and maintenance time are respectively equal to 12,454 and 186. For fabrication machine 1, the maintenance level decreases from 595.28 to 15.28 (after completing product 18, the maintenance level is 15.28) linearly; then a PM activity is performed resulting in the maintenance level reverting to the initial value 595.28; finally the maintenance level continues to decrease linearly until all products are completed. For the assembly machine (machine 3), there is a slight difference in the trend of maintenance levels compared

Table 2 The processing and assembly times of illustrative example

j	t_{j1}	t_{j2}	p_j	j	t_{j1}	t_{j2}	p_j	j	t_{j1}	t_{j2}	p_j	j	t_{j1}	t_{j2}	p_j
1	23	8	23	6	5	23	94	11	65	71	91	16	59	87	49
2	64	35	46	7	52	67	69	12	80	29	96	17	75	68	34
3	35	6	92	8	96	79	37	13	64	42	34	18	64	100	25
4	69	40	98	9	23	15	93	14	81	93	21	19	40	82	92
5	63	57	25	10	84	12	47	15	63	100	26	20	20	32	96

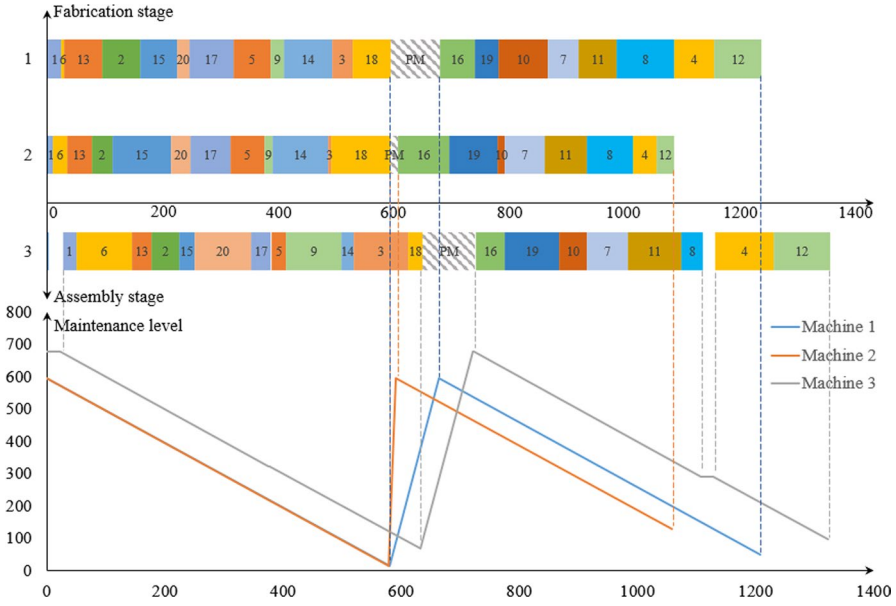

Fig. 1 Gantt chart and maintenance level curve with the optimal solution

with fabrication machines. In fabrication machines no idle time happens; while in assembly machine it will generate idle time when C_{ik} is larger than $C_{i-1,m+1}$. The maintenance levels remain constant during idle time, i.e., the period time before products 1 and 4 on the assembly machine. Hence, each machine executes PM once and the total maintenance time is 186 (84 + 13 + 89). The final objective value is 12640 (total completion time 12,454 + total maintenance time 186).

4 Constructive heuristics

Two-stage assembly flow shop scheduling is proved to be an NP-hard problem (Potts et al. 1995), and further the consideration of PM enhances its NP-hardness. Given the complexity of $DP_m \rightarrow 1|PM|$, this paper aims to improve all the current constructive heuristics and proposes a new local search-based constructive heuristic (LSC) to solve this new problem quickly and efficiently. The constructive heuristics of literature main include TCK1 and TCK2 by Tozkapan et al. (2003), S1, S2 and S3 by Al-Anzi and Allahverdi (2006), A1 and A2 by Al-Anzi and Allahverdi (2006), G1, G2, G3 and G4 by Lee (2018), FAP by Framinan and Perez-Gonzalez (2017), NEH by Fernandez-Viagas and Framinan (2014) and processing-time-based pairwise exchange heuristic (PPE) by Sung and Kim (2008). To adapt these heuristics to the integration problem, a latest PM decision strategy is proposed and embedded into constructive heuristics.

In the latest PM decision strategy, as shown in Fig. 2, before processing each product, it needs to be judged whether a PM activity is performed immediately. If the

Heuristic: Latest PM decision strategy

For $k \leftarrow 1$ to $m+1$ do
  $ML_{0k}^{a} \leftarrow ml_{k}^{0}$;
  For $i \leftarrow 1$ to n do
    If $ML_{i-1,k}^{a} - \mu_k \cdot \sum_{j=1}^{n} t_{jk}(or\ p_j) \cdot X_{ji} < 0$ then //p_j is used in assembly stage
      $Y_{ik} \leftarrow 1$; //a PM is performed
      $ML_{ik}^{a} \leftarrow ml_{k}^{0} - \mu_k \cdot \sum_{j=1}^{n} t_{jk}(or\ p_j) \cdot X_{ji}$; //$p_j$ is used in assembly stage
    Else:
      $Y_{ik} \leftarrow 0$;
      $ML_{ik}^{a} \leftarrow ML_{i-1,k}^{a} - \mu_k \cdot \sum_{j=1}^{n} t_{jk}(or\ p_j) \cdot X_{ji}$; //$p_j$ is used in assembly stage
    End If
  End For
End For

Fig. 2 Latest PM decision strategy

maintenance level after processing this product is less than 0, a PM activity is needed before this product to avoid product halts. Through this strategy, we can prevent premature PM execution and take advantage of the maintenance level of each machine. Further, the maintenance time has also been optimized by reducing maintenance times. The adapted versions of the constructive heuristics are respectively named TCK1_PM, TCK2_PM, S1_PM, S2_PM, S3_PM, A1_PM, A2_PM, G1_PM, G2_PM, G3_PM, G4_PM, FAP_PM, NEH_PM, PPE _PM and LSC_PM.

4.1 TCK1_PM and TCK2_PM

The original TCK1 constructs $m+1$ product sequences (each sequence is denoted as $\Pi_s = \{\pi_{s1}, \ldots, \pi_{si}, \ldots, \pi_{sn}\}$) by sorting the products in non-descending order of fabrication or assembly time. The sequence with the lowest objective value is selected. Applied to our new problem, TCK1_PM first obtains $m+1$ product sequences with the same method, and then use the proposed latest PM decision strategy to determine the PM time points in each machine. Finally, the sequence with the lowest total completion time and total maintenance time is selected.

The TCK2_PM first constructs three product sequences by sorting the products in the non-descending order of three indices respectively. These indices are: $MPT_j = min\{t_{j1}, t_{j2}, \ldots, t_{jm}, p_j\}$, $APT_j = \frac{\sum_{k=1}^{m} t_{jk} + p_j}{m+1}$ and $MXPT_j = max\{t_{j1}, t_{j2}, \ldots, t_{jm}, p_j\}$. Analogous to TCK1_PM, TCK2_PM determines PM activities according to the latest PM decision strategy, and selects the sequence with the lowest objective value.

4.2 S1_PM, S2_PM and S3_PM

Heuristic S1_PM first sorts the products in non-descending order of assembly time p_j; heuristic S2_PM first sorts the products in non-descending order of $\max_{1 \le k \le m} t_{jk}$; heu-

ristic S3_PM first sorts the products in non-descending order of $\max_{1\le k\le m} t_{jk} + p_j$. After obtaining the product sequence, we use the latest PM decision strategy to determine the PM time points and finally calculate the total completion time and total maintenance time.

4.3 A1_PM and A2_PM

Distinct from the constructive heuristics mentioned above, A1_PM and A2_PM construct a product sequence taking into account products to be assigned and all assigned products. These two heuristics construct a sequence by iteratively inserting the non-assigned products in the end. Clearly, at iteration i, A1_PM evaluates each candidate product based on the fabrication time of this product and that of the assigned products. The indicator of each candidate product $A1_j$ can be calculated by the expression $\max_{1\le k\le m}\left(\sum_{r\in J_{as}} t_{rk} + t_{jk}\right)$ where J_{as} is the set of assigned products. The candidate product with the lowest value of $A1_j$ is assigned to insert to the end of the existing sequence. As for A2_PM, it has the identical procedure to A1_PM, but additionally considers assembly time to evaluate each candidate product. The indicator of each candidate product $A2_j$ is equal to $\max_{1\le k\le m}\left(\sum_{r\in J_{as}} t_{rk} + t_{jk}\right) + p_j$. After constructing a product sequence, the latest PM decision strategy is used to determine the PM time points.

4.4 G1_PM, G2_PM, G3_PM and G4_PM

G1_PM, G2_PM, G3_PM and G4_PM construct a sequence based on the completion time of the product to be assigned and that of the latest assigned product in the fabrication and assembly stages. Specifically, G1_PM evaluate each unassigned product with a difference value between the completion time of this product and that of the latest assigned product in the assembly stage; G2_PM considers the difference value between the completion time of this product and that of the latest assigned product in the fabrication stage; G3_PM allows for the difference value between the completion time of this product in fabrication stage and that of the latest assigned product in the assembly stage; G4_PM considers the difference value between the completion time of this product in the assembly stage and that of the latest assigned product in the fabrication stage.

Taking G1_PM for example, at iteration i, it is assumed that each unassigned product j is allocated to the i-position of the sequence. Then the evaluation value of product j can be calculated by the expression $A_{ji}\left[C_{i,m+1}\right] - C_{i-1,m+1}$ ($\max_{1\le k\le m}\left(A_{ji}\left[C_{ik}\right]\right) - \max_{1\le k\le m}\left(C_{i-1,k}\right)$ in G2_PM, $\max_{1\le k\le m}\left(A_{ji}\left[C_{ik}\right]\right) - C_{i-1,m+1}$ in G3_PM and $A_{ji}\left[C_{i,m+1}\right] - \max_{1\le k\le m}\left(C_{i-1,k}\right)$ in G4_PM). Among them, $A_{ji}[x]$ is the value of x when product j is assumed to be assigned in position i. Note that when calculating $A_{ji}\left[C_{ik}\right]$ $(k = 1, \ldots, m+1)$, the latest PM decision strategy should also be used to judge whether a PM activity is performed before the start of this product. Finally,

the unassigned product with the lowest evaluation value is selected and inserted in the i-position of the sequence, and the corresponding PM decision is reserved.

4.5 FAP_PM

FAP_PM constructs a sequence by iteratively appending the unassigned products at the end. It takes full account of the dominance between the fabrication and assembly stages. Similar to the constructive heuristics in Sects. 4.3 and 4.4, this heuristic selects the product with the lowest value of new indicator from the set of unassigned products at iteration i. This new indicator is denoted as FAP_{ji}. It is assumed that the unassigned product j is assigned to position i. If the completion time of this product in the fabrication stage C_{ji}^{1} is larger than that of the product at position $i-1$ in the assembly stage C_{2}^{*}, $FAP_{ji} = C_{ji}^{1} + \frac{(n-i+1)}{n} \cdot \left(C_{1\blacksquare} + \frac{p_{\blacksquare}}{m} \right)$; otherwise, $FAP_{ji} = p_j + \frac{(n-i+1)}{n} \left(C_{1\blacksquare} + \frac{p_{\blacksquare}}{m} \right)$. Among them, specific calculations of $C_{1\blacksquare}$ and $p_{\blacksquare}$ refer to Framinan and Perez-Gonzalez (2017). Applied in our new problem, FAP_PM uses the latest PM decision strategy to determine the PM time points in each machine when calculating the completion time.

4.6 NEH_PM and PPE_PM

NEH_PM mainly consists of two phases. The first phase generates a seed product sequence according to the descending sum of the total fabrication and assembly times $\sum_{k=1}^{m_1} t_{jk} + p_j$. In the second phase, the product in the first position of the seed product sequence is placed in the first position of a partial sequence. Then at iteration i ($i = 2, \ldots, n$), the product in position i of the seed product sequence is inserted into all possible positions of the partial sequence. Thus i partial sequences are generated where the sequence with the minimum objective value is selected for the next iteration. After finishing the n-1-th iteration, a complete product sequence is obtained.

PPE _PM also has two phases. The first phase is the same as that of NEH_PM. In the second phase, each product in position i ($i = 1, \ldots, n$-1) of the current sequence is exchanged with all subsequent products to generate n-1 sequences. The sequence with the minimum objective is selected as the current sequence. Repeat the above procedure until all the positions have been traversed. Note that the latest PM decision strategy is also used to determine the PM execution points when calculating the objective of each partial or complete sequence.

4.7 LSC_PM

This paper proposes a new local search-based constructive heuristic (LSC_PM) to solve $DP_m \rightarrow 1|PM| \sum_{i=1}^{n} C_{i,m+1} + \sum_{i=1}^{n} \sum_{k=1}^{m+1} Y_{ik} \cdot dpm_k$ problem. LSC_PM combines the swap and local search techniques. It aims at achieving a better product

sequence by testing each position iteratively. It mainly includes two phases. Same as the NEH_PM, The first phase generates an initial product sequence according to the descending sum of the total fabrication and assembly times $\sum_{k=1}^{m_1} t_{jk} + p_j$. In the second phase, For each product in position i ($i=n$, n-1, …, 2) of the sequence, it will be exchanged with all other products in position s ($s=i$-1, i-2, …, 1) to create a set of candidate sequences. If the best candidate sequence improves the current sequence i.e. the objective value of the best candidate solution is better than that of the current solution, it replaces the current sequence and the position i is reset as n. Repeat the above process until all swaps have been involved.

In the above heuristics, the complexity of TCK1_PM is $O(n \cdot m^2)$; those of TCK1_PM, S1_PM, S2_PM, S3_PM, A1_PM and A2_PM are $O(n \cdot m^2)$; those of G1_PM, G2_PM, G3_PM, G4_PM, FAP_PM, NEH_PM and PPE_PM are $O(n \cdot m^2)$; that of LSC_PM is between $O(n \cdot m^2)$ and $O(n \cdot m^2)$ since the position may be reset as n.

5 Meta-heuristics

The previous constructive heuristics can obtain a solution very fast and efficiently; while there is still a certain gap between the obtained and optimal solutions. It is necessary to develop meta-heuristics to get the solution with great quality. To our knowledge, there is no available meta-heuristic focusing on our new problem. Hence this section designs seven meta-heuristics including three variants of iterated local search (ILS), three variants of Q-learning-based ant colony system with local search (QACS_LS) and a Q-learning-based hyper-heuristics (QHH). The performances of these meta-heuristics are discussed in Sect. 6.5 to select the best meta-heuristic. Note that in these meta-heuristics the proposed latest PM decision strategy is used to determine the PM time points and calculate the objective value when a product sequence is found. The details of the proposed meta-heuristics are as follows.

5.1 Iterated local search (ILS)

Since the iterated local search is proved to be useful in tackling regular two-stage assembly flow shop scheduling (Framinan and Perez-Gonzalez 2017), this paper designs three versions of ILS for addressing our new problem. The original ILS starts with an initial solution usually generated by a constructive heuristic, and then iteratively improves this solution by two phases: local search and perturbation phases. As we all know, the local search aims to improve the performance by exploring the neighborhood, but it is easy to fall into local optima. In this situation, the perturbation phase can help the methods escape from local optima. In this paper, we design three local search types and two perturbation types to improve the performance of ILS.

The first local search labeled LS1 mainly adopts the second phase of LSC_PM to create new neighboring solutions. The flowchart of this local search is shown in Fig. 3.

The second local search labeled LS2 uses forward insertion to complete the neighborhood search. Clearly, for each product in position i ($i=n, n\text{-}1, \ldots, 2$) of the current solution, it is inserted into all possible position before its original position i.e. set position $s=i\text{-}1, i-2, \ldots, 1$, to create a set of neighboring candidate solutions. If the best candidate solution improves the current solution, it replaces the current solution and the position i is reset as n. Repeat the above process until all forward insertions have been involved. The flowchart of this local search is shown in Fig. 4.

The third local search labeled LS3 employs backward insertion to exploit all the neighborhoods. Specifically, for each product in position i ($i=1, 2, \ldots, n-1$) of the current solution, it is inserted into all possible positions after its original position i.e. set position $s=i+1, i+2, \ldots, n$, to create a set of neighboring candidate solutions. If the best candidate solution improves the current solution, it replaces the current solution and the position i is reset as 1. Repeat the above process until all backward insertions have been involved. The flowchart of this local search is shown in Fig. 5.

Perturbation is similar to the restart phase (Minella et al. 2011; Zhang et al. 2020a) but applied in simple-objective problems. The first perturbation labeled Per1 uses the swap to disturb the current solution slightly to increase the exploration

Fig. 3 Local search 1

Procedure: LS1 on $\Pi = \{\pi_1, \ldots, \pi_i, \ldots, \pi_n\}$

1: For $i \leftarrow n$ to 2 do
2: $\Omega \leftarrow \Phi$;
3: For $s \leftarrow i-1$ to 1 do
4: $\Pi^* \leftarrow \Pi$;
5: $\pi_i^* \leftarrow \pi_s$;
6: $\pi_s^* \leftarrow \pi_i$;
7: $\Omega \leftarrow \Omega \cup \Pi^*$;
8: End For
9: $\Pi^1 \leftarrow$ Π^* with the best Obj in Ω;
10: If $Obj(\Pi^1) < Obj(\Pi)$ then
11: $\Pi \leftarrow \Pi^1$;
12: $i \leftarrow n$;
13: End If
14: End For
15: Return Π.

Fig. 4 Local search 2

Procedure: LS2 on $\Pi = \{\pi_1, \ldots, \pi_i, \ldots, \pi_n\}$

For $i \leftarrow n$ to 2 do
  $\Omega \leftarrow \Phi$;
  For $s \leftarrow i-1$ to 1 do
    $\Pi^* \leftarrow \Pi$;
    $\pi_s^* \leftarrow \pi_i$;
    For $k \leftarrow s+1$ to i do
      $\pi_k^* \leftarrow \pi_{k-1}$;
    End for
    $\Omega \leftarrow \Omega \cup \Pi^*$;
  End For
  $\Pi^1 \leftarrow$ Π^* with the best Obj in Ω;
  If $Obj(\Pi^1) < Obj(\Pi)$ then
    $\Pi \leftarrow \Pi^1$;
    $i \leftarrow n$;
  End If
End For
Return Π.

space. In every perturbation, two products are randomly selected and their positions are exchanged. Referring to Framinan and Perez-Gonzalez (2017), the perturbation times d is determined by the improvement times and the number of products. The initial perturbation times is set 1. If the best product sequence has not been improved for n times, the perturbation times d is equal to the minimum of $d+1$ and $n-1$. Once the best product sequence is improved, the initial perturbation is return 1. Different from Per1, perturbation Per2 uses the insertion to disturb the current solution. In every perturbation, a product in position i ($i=1, 2, \ldots, n-1$) is randomly removed and reinserted in the position $i+1$.

After a preliminary experiment, we find that if we adopt the same operator in local search and perturbation phases i.e. swap in local search and perturbation (LS1 and Per1), it is easy to make the algorithm fall into local optima. In other words, the perturbation does not work. Hence through combining the different phases, we extend three versions: ILS1_Per2 (LS1 and Per2), ILS2_Per1 (LS2 and Per1) and ILS3_Per1 (LS3

Fig. 5 Local search 3

Procedure: LS3 on $\Pi = \{\pi_1, \ldots, \pi_i, \ldots, \pi_n\}$

For $i \leftarrow 1$ to n-1 do
  $\Omega \leftarrow \Phi$;
  For $s \leftarrow i + 1$ to n do
    $\Pi^* \leftarrow \Pi$;
    $\pi_s^* \leftarrow \pi_i$;
    For $k \leftarrow s - 1$ to i do
      $\pi_k^* \leftarrow \pi_{k+1}$;
    End for
    $\Omega \leftarrow \Omega \cup \Pi^*$;
  End For
  $\Pi^1 \leftarrow$ Π^* with the best Obj in Ω;
  If $Obj(\Pi^1) < Obj(\Pi)$ then
    $\Pi \leftarrow \Pi^1$;
    $i \leftarrow n$;
  End If
End For
Return Π.

and Per1). To accelerate the convergence of ILS, three versions of ILS start with a solution obtained by the constructive heuristic LSC_PM proved to be the best constructive heuristic in Sect. 6.4. Then, this solution is iteratively improved by local search and perturbation phases. When the termination condition is reached, the best solution is output. The flowcharts are presented in Figs. 6, 7.

Meta-heuristic: ILS1_Per2

$\Pi^{current}$ obtained by heuristic LSC_PM; //set the initial solution
$\Pi^{best} \leftarrow \Pi^{current}$;
$d \leftarrow 1$; //set the initial perturbation times
$improvement \leftarrow 0$; //set the counter without improvement
When *the termination condition is not reached* do
    $\Pi^{current} \leftarrow LS1(\Pi^{current})$;
    If $Obj(\Pi^{current}) < Obj(\Pi^{best})$ then
        $\Pi^{best} \leftarrow \Pi^{current}$; //update the best solution
        $improvement \leftarrow 0$; //reset the counter
        $d \leftarrow 1$; //reset the perturbation times
    Else
        $improvement \leftarrow improvement + 1$;
    End If
    If $improvement > n$ then
        $d \leftarrow \min\{d + 1, n - 1\}$;
        $improvement \leftarrow 0$; //reset the counter
    End If
    For $k \leftarrow 1$ to d do //execute Per2 on $\Pi^{current}$
        Randomly insert the product in position *i* to position *i*+1;
    End For
End When
Return Π^{best}.

Fig. 6 The flowchart of ILS1_Per2

Meta-heuristic: ILS2_Per1 and ILS3_Per1

$\Pi^{current}$ obtained by heuristic LSC_PM; //set the initial solution
$\Pi^{best} \leftarrow \Pi^{current}$;
$d \leftarrow 1$; //set the initial perturbation times
$improvement \leftarrow 0$; //set the counter without improvement
When *the termination condition is not reached* do
    $\Pi^{current} \leftarrow LS2(\Pi^{current})$; //used in ILS2_Per1
    $\Pi^{current} \leftarrow LS3(\Pi^{current})$; //used in ILS3_Per1
    If $Obj(\Pi^{current}) < Obj(\Pi^{best})$ then
        $\Pi^{best} \leftarrow \Pi^{current}$; //update the best solution
        $improvement \leftarrow 0$; //reset the counter
        $d \leftarrow 1$; //reset the perturbation times
    Else
        $improvement \leftarrow improvement + 1$;
    End If
    If $improvement > n$ then
        $d \leftarrow \min\{d + 1, n - 1\}$;
        $improvement \leftarrow 0$; //reset the counter
    End If
    For $k \leftarrow 1$ to d do //execute Per1 on $\Pi^{current}$
        Randomly swap two products in positions *i* and *s*;
    End For
End When
Return Π^{best}.

Fig. 7 The flowchart of ILS2_Per1 and ILS3_Per1

5.2 Q-learning-based ant colony system with local search (QACS_LS)

This paper designs three memetic versions of the ant colony system (ACS) with a local search procedure to conveniently tackle this new problem. ACS starts with an ant colony with a predefined number. Each ant tries to select a product in each iteration according to the global pheromone and heuristic information to construct a complete product sequence. Sequentially, the global pheromone and the best solution are updated. In ACS four parameters are predetermined, specifically including the population size PS, parameters α and β determining the importance of pheromone and heuristic information, and parameter q_0. In our proposed QACS_LS, the local search procedures are used in conjunction to ACS to enhance its neighbor search ability; Q-learning is first designed to determine an appropriate parameter combination in each iteration. Hence QACS_LS mainly contains six phases: initiation, Q-learning-based parameter selection, ant colony release, pheromone release, best solution update and local search. Since three local search procedures are proposed in this paper, three variants are respectively named QACS_LS1, QACS_LS2 and QACS_LS3. The details of the six phases are as follows.

5.2.1 Initiation

The initiation phase aims to determine the global pheromone τ_{ur} which is used to lead the ants to select a product in each iteration. To get an initial global pheromone with great quality, the proposed meta-heuristic employ the 15 above mentioned constructive heuristics to generate the initial colony, so that the population size is set 15. Based on the obtained product sequences, the global pheromone is used to record the times of every selected product during each iteration. Specifically, in the beginning, the initial value of each τ_{ur} is set 0. Then, for every initial solution, if product u is selected in the r-th iteration, the corresponding τ_{ur} pluses 1. According to the above procedure, the global pheromone is got.

Besides, the current state of Q-learning is at start and the values in Q-value tables are set 0. These will be introduced in the following.

5.2.2 Q-learning-based parameter selection

In our proposed QACS_LS we design Q-learning-based parameter selection to replace the parameter calibration to identify an appropriate parameter combination before every ant colony release. Q-learning is the science of making the best decisions, and is a dynamic programming method based on numerical iteration. It involves four elements: state, action, Q-value table and reward function. Generally, Q-learning uses a greedy strategy to select an action with the greatest Q value based on the current state, and then determines the next state and updates the Q-value table after receiving the reward. It updates the Q-value table according to the Behrman equation calculated by Eq. (25). In this equation, $newQ_{S,A}$ and $Q_{S,A}$ are respectively the updated and current Q-values of current state S and action A; α and γ are learning rate and discount factor; $R_{S,A}$ is the reward value after taking action A under the state S; $max\tilde{Q}_{S,A}$ means the maximum Q-value in

the next current state. Regarding α, if it is equal to 1.0, that means the new information is the only one clue; while if it is 0.0, that means the method is not going to learn anything. We believe the new and old information is of equal importance and hence α is set 0.5. As for γ, it usually sets 0.9.

$$newQ_{S,A} = (1-\alpha)\cdot Q_{S,A} + \alpha\cdot\left(R_{S,A} + \gamma\cdot\left(max\tilde{Q}_{S,A}\right)\right) \tag{25}$$

In our proposed Q-learning parameter selection, we develop three states: start, no-improvement and improvement. The method is at start state at the beginning. If the best solution is improved at ant colony release phase, the next state turns to improvement; otherwise, it is at no-improvement state. The actions are the different levels of parameters. Since there are three parameters (α, β and q_0) involving in QACS_LS, we develop three Q-value tables to store the Q-value of different levels. The regular levels are all used here to represent actions. Clearly, parameter α is at 8 levels: 0.05, 0.15, 0.25, 0.5, 0.75, 0.8, 0.85 and 0.9; parameter β is at 3 levels: 1, 2 and 3; parameters q_0 is at 5 levels: 0.15, 0.25, 0.35, 0.5 and 0.85. These values come from the research by Zhang et al. (2020b). Hence the Q-value tables are respectively represented as $Q^{\alpha}_{3\times 8}$, $Q^{\beta}_{3\times 3}$ and $Q^{q_0}_{3\times 5}$. Regarding reward function in ACS_LS, for each solution in the colony, if it improves the best solution, the reward value pluses 1; otherwise, the reward value minus 1. Note that the initial values of Q-value tables are set 0.

In a word, we need to select the parameter level (action) with the greatest Q value before executing the ant colony release. Take parameter α as an example, $A \leftarrow \arg\max_{A} Q^{\alpha}_{current,A}$. After a colony is generated, according to the improvement of the best solution, the Q-value tables and current state are updated.

5.2.3 Solution generation and ant colony release

In the proposed QACS_LS algorithm, each ant finds a product sequence in a way that constraints (2–3) should be satisfied. In each iteration r, ant must select a product u from the unassigned product set according to the global pheromone and heuristic information. Hence a total of n iteration are involved. The procedure is detailed below.

Step 1: $r=1$ and unassigned product set $J_{un} = \{1, \ldots, n\}$;

Step 2: Select a product.

A product is selected from the unassigned set according to the ant colony system state transition rule. This rule considers the global pheromone variable in r-th iteration and the heuristic information. In our paper, the smallest processing time rule (*SPT*) is used to determine the priority value. In the unassigned product set, the product with the largest average fabrication and assembly time has the smallest priority value 1. The second one has the value 2. By analogy, all the unassigned products are given a priority value. The rule determines a product using Eq. (26).

$$i = \begin{cases} \arg\max\limits_{u \in C_r^s} \left\{ \left[\tau_{ur}\right]^{\alpha} \cdot \left[\eta_u\right]^{\beta} \right\} & \text{if } q \le q_0 \\ \arg\max\limits_{u \in C_r^s} \Pr(u, r) & \text{if } q > q_0 \end{cases} \tag{26}$$

where q is a randomly generated number between [0, 1]; η_u is the heuristic information value of product u; $Pr(u, r)$ is the selection probability of product u in the r-th iteration, which can be calculated using Eq. (27).

$$\Pr(u, r) = \frac{\left[\tau_{ur}\right]^{\alpha} \cdot \left[\eta_u\right]^{\beta}}{\sum\limits_{h \in C_r^s} \left[\tau_{hr}\right]^{\alpha} \cdot \left[\eta_h\right]^{\beta}} \tag{27}$$

where τ_{hr} and η_h are similar to τ_{ur} and η_u, but they are related to the product h. According to this rule, there are two behaviors to select the product from the unassigned product set:

1. If $q \le q_0$, the algorithm tends to select a product with the maximum value of joint global pheromone and heuristic information value;
2. If $q > q_0$, the algorithm determines a product based on probability, which may make the algorithm choose a non-explored path.

Step 3: Update local pheromone $\Delta\tau_{ur}^s$ and unassigned product set J_{un}.

After a product u is selected in the r-th iteration by ant s, the value of the local pheromone $\Delta\tau_{ur}^s$ is updated and equal to 1. And this product is removed from the set J_{un}.

Step 4: Evaluate the termination condition.

If the unassigned product set is empty, the procedure is terminated and a complete solution is obtained; otherwise, $r = r + 1$ and return to Step 2.

Through the above solution generation, PS solutions are obtained to construct a colony.

5.2.4 Pheromone release and best solution update

In the above section, a colony is obtained along with the local pheromone $\Delta\tau_{ur}^s$. The local pheromone is used to update the global pheromone. According to the pheromone release strategy by Baykasoglu and Dereli (2008), we update the global pheromone quantity from two aspects: deposition and evaporation. The deposition aims to perform the trails of all ants and updates the pheromone using the Eq. (28).

$$\tau_{ur} \leftarrow \tau_{ur} + \Delta\tau_{ur}^s \tag{28}$$

However, the trails of some ants who build the solutions with worse performance are also stored in the deposition. This situation will send bad messages to the ants in the next iteration and further induce them to explore bad paths. Hence, the evaporation tries to release these trails to avoid this situation. It first calculates the average

total completion time (*atct*) and average total maintenance time (*atmt*) in the current colony. And then, if the total completion time and total maintenance time of the solution obtained by ant *s* are larger than or equal to *atct* and *atmt* respectively, the global pheromone trails are evaporated by Eq. (29).

$$\tau_{ur} \leftarrow \tau_{ur} - \Delta\tau_{ur}^{s} \tag{29}$$

In the ant colony release, when a new solution is generated, it will compare with the best solution. If the new solution improves the best one, it will replace it.

The flowcharts of QACS_LS1, QACS_LS2 and QACS_LS3 are shown in Fig. 8.

Meta-heuristic: QACS_LS1, QACS_LS2 and QACS_LS3

Generate $\Pi^i\ (i = 1,2,\ldots,PS)$ by 15 constructive heuristics; //initialize colony
$\Pi^{best} \leftarrow \arg\min_{\Pi^i}\{Obj(\Pi^i)\}$;
Update τ_{ur} by $\Pi^i\ (i = 1,2,\ldots,PS)$; // initialize global pheromone
$State_{current} \leftarrow start$; //initialize the current state of Q-learning
Initialize Q-value tables;
When *the termination condition is not reached* do
    Q-learning-based parameter selection; //select the parameter combination
    Regenerate $\Pi^i\ (i = 1,2,\ldots,PS)$ by ant colony release;
    Update Q-value tables and current state;
    Pheromone release; //update τ_{ur} by $\Delta\tau_{ur}^{i}$
    For $i \leftarrow 1$ to PS do //update the best solution
        If $Obj(\Pi^{best}) < Obj(\Pi^i)$ then
            $\Pi^{best} \leftarrow \Pi^i$;
        End If
    End For
    $\Pi^{best} \leftarrow LS1(\Pi^{best})$; //used in QACS_LS1
    $\Pi^{best} \leftarrow LS2(\Pi^{best})$; //used in QACS_LS2
    $\Pi^{best} \leftarrow LS3(\Pi^{best})$; //used in QACS_LS3
End When
Return Π^{best}.

Fig. 8 The flowchart of QACS_LS1, QACS_LS2 and QACS_LS3

5.3 Q-learning-based hyper-heuristics (QHH)

Recently, since hyper-heuristics (HH) combines the advantages of heuristics and meta-heuristics, it has been widely extended to tackle combination optimization problems such as flow shop and job shop scheduling. HH mainly comprises of high and low level heuristics and a move acceptance. The high level heuristic is not related to the problem domain and aims to select a specific low level heuristic according to some rules. The low level heuristics can be regarded as some heuristic rules or neighborhood search operators with problem characteristics to generate a new solution. The move acceptance decides the acceptability of the new solution.

Our proposed QHH starts with an initial solution generated by constructive heuristic LSC_PM. In the main QHH loop, Q-learning-based selection first decides a low level heuristic to generate a new solution; then a late acceptance is used to judge whether the new solution replaces the current one; finally the Q-value table and the best solution is updated. The procedure of QHH is detailed below.

Q-learning-based selection: is the same as the Q-learning-based parameter selection in the above section; while the low level heuristics instead of parameter levels represent actions. Similarly, when a new solution is generated, the Q-value table and current state are updated according to the improvement of the best solution.

Problem-specific low level heuristics: this section proposes 15 low level heuristics to improve the quality of the best solution.

LLH1: randomly selects two products from the sequence and exchanges their positions.

LLH2: randomly selects a product and then inserts it into any position before its original position.

LLH3: randomly selects a product and then inserts it into any position after its original position.

LLH4: executes LLH1 and LLH2 in turn.

LLH5: executes LLH2 and LLH1 in turn.

LLH6: executes LLH1 and LLH3 in turn.

LLH7: executes LLH3 and LLH1 in turn.

LLH8: executes LLH2 and LLH3 in turn.

LLH9: executes LLH3 and LLH2 in turn.

LLH10: executes LLH1, LLH2 and LLH3 in turn.

LLH11: executes LLH1, LLH3 and LLH2 in turn.

LLH12: executes LLH2, LLH3 and LLH1 in turn.

LLH13: executes LLH2, LLH1 and LLH3 in turn.

LLH14: executes LLH3, LLH2 and LLH1 in turn.

LLH15: executes LLH3, LLH1 and LLH2 in turn.

Late acceptance: is first proposed by Bykov and Yuri (2008). It decides the acceptability of the new solution by two conditions that the new solution improves the best one in the current iteration or L iteration earlier. Hence a circular queue of size L is implemented to store the best solutions in the previous L iterations. In QHH, the size L is set the number of products n.

The flowcharts of QHH are shown in Fig. 9.

Meta-heuristic: QHH

$\Pi^{current}$ obtained by heuristic LSC_PM; //set the initial solution

$\Pi^{best} \leftarrow \Pi^{current}$;

$State_{current} \leftarrow start$; //initialize the current state of Q-learning

Initialize Q-value tables;

Initialize the circular queue; // put Π^{best} in queue

When *the termination condition is not reached* do

    $r \leftarrow$ Q-learning-based selection; //select the low heuristic

    $\Pi^{current} \leftarrow LLHr$;

    Update Q-value tables and current state;

    If $Obj(\Pi^{best}) < Obj(\Pi^{current})$ then //update the best solution

        $\Pi^{best} \leftarrow \Pi^{current}$;

    End If

    Late acceptance on $\Pi^{current}$;

    Update the circular queue;

End When

Return Π^{best}.

Fig. 9 The flowchart of QHH

6 Experimental study

In this section, we have carried out three sets of experiments to verify the performance of the proposed MILP model, constructive heuristics and meta-heuristics. First, we compare our MILP model with the regular model about two-stage assembly flow shop scheduling to illustrate the significance of integration optimization of two-stage assembly flow shop scheduling and PM. Then, we conduct a set of comparison experiments between the MILP model and constructive heuristics to test the performance of these heuristics in small-scale instances. Next, we compare these constructive heuristics in large-scale instances to further explore their performance. Finally, we compare the proposed 7 meta-heuristics with 5 existing meta-heuristics to measure their performance.

To do so, we generate a set of benchmark instances for our new integration optimization problem. According to Allahverdi and Al-Anzi (2009), the benchmark consists of 30 replications of instances for different combinations of n and m where $n \in \{20, 40, 60, 80, 100, 120\}$ and $m \in \{2, 4, 6, 8\}$. Hence a total of 720 benchmark instances are generated. In each instance, the processing times on fabrication and assembly machines are drawn from a uniform distribution [1, 100]. Further, according to Ruiz et al. (2007), the set of scale parameter θ is related to the number of products, and therefore in our paper $\theta = 1500, 1900, 2300, 2700, 3100$ and 3500 for $n = 20, 40, 60, 80, 100$ and 120 respectively; the shape parameter β is randomly selected from {2, 3, 4}; t_p is set at 1 and t_r at 8 for all the experiments; the maintenance duration dpm_k is 25%, 50%, 75%, 100% or 150% of the average fabrication (assembly) time each machine k. Hence based on the above generation, a total of 720 benchmark instances with scheduling and maintenance parameter involved are employed to conduct the experimental computational.

Note that all the constructive heuristics and meta-heuristics are coded in the C++ programming language and are run on a computer with Intel(R) Core(TM) i5 4440 processor running at 2.80 GHz and with 2.00 GBytes of main RAM memory.

6.1 Performance indicators

Two evaluation indicators are used to evaluate the effectiveness and efficiency of the improved constructive heuristics and meta-heuristics. The first one is the relative percentage deviation (RPD) and can be calculated by Eq. (30). This indicator shows how far the solution of a heuristic is from the best one found for a given problem (Miyata et al. 2019), and is usually used to evaluate the effectiveness of the methods.

$$RPD_{heuristic} = \frac{Obj_{heuristic} - Obj_{best}}{Obj_{best}} \times 100 \tag{30}$$

where $Obj_{heuristic}$ is the objective value obtained by heuristic and Obj_{best} is the minimum objective value founded by all heuristics. When each RPD is obtained, the average relative percentage deviation (ARPD) is calculated according to:

$$ARPD_{heuristic} = \frac{\sum_{\forall s} RPD_{s,heuristic}}{S} \tag{31}$$

where S means the number of instances for a given class.

The second one is the average CPU time (ACPU) which is used to evaluate the efficiency. It can be calculated by Eq. (32).

$$ACPU_{heuristic} = \frac{\sum_{\forall s} CPU_{s,heuristic}}{S} \tag{32}$$

where $CPU_{s,heuristic}$ is the run time by a heuristic to generate a solution, and S is the number of instances for a given class.

6.2 The significance of integration optimization of scheduling and PM

In this section, a computational experiment is conducted to demonstrate the significance of two-stage assembly flow shop scheduling and PM. The example in Sect. 3.3 is revisited here. In the first scene, this example is solved through the proposed mathematical model, coded by the IBM Cplex. The final optimal solution is shown in Fig. 1. In the second scene, the example is solved through the regular two-stage assembly flow shop scheduling model with the total completion time objective, proposed by Seidgar et al. (2013). This model is also coded by the IBM Cplex, and hence a product sequence is obtained. Based on this sequence, the latest PM decision strategy is employed here to insert the PM activities into the sequence. The final solutions without and with PM activities in the second scene are shown in Fig. 10.

In the first scene, the final product sequence is {1, 6, 13, 2, 15, 20, 17, 5, 9, 14, 3, 18, 16, 19, 10, 7, 11, 8, 4, 12}; and the total completion time and total maintenance time are respectively equal to 12,454 and 186. In the second scene, the optimal sequence obtained by the regular model is {1, 6, 13, 2, 15, 20, 17, 5, 9, 14, 3, 10, 7, 18, 19, 16, 4, 8, 11, 12}; regarding the solution without PM activities, the total completion time is 11833; while when PM activities are involved, PM activities will be performed on machine 1 twice, machine 2 once and machine 3 once, therefore the total completion time and total maintenance time are respectively equal to 13,173 and 270. Comparing two scenes, the total completion time achieved by the proposed MILP model is reduced by 719, about 5.77%; similarly, the total maintenance time is reduced by 84, about 45.16%. These results suggest that the integration optimization of two-stage assembly flow shop scheduling and PM can help reduce the total completion time and total maintenance time. Hence the proposed integration optimization of scheduling and PM is significantly necessary for a two-stage assembly flow shop.

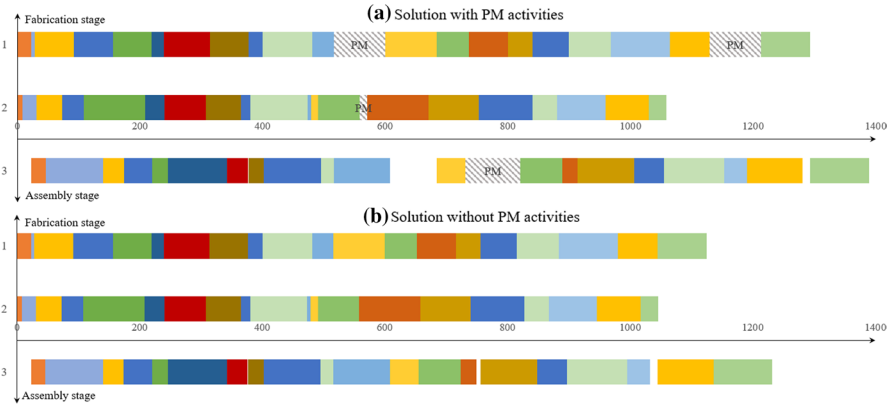

Fig. 10 The final solutions with and without PM activities in the second scene

Table 3 Comparison results for small-scale instances between model and heuristics

Case	Model	TCK1_PM	TCK2_PM	A1_PM	A2_PM	S1_PM	S2_PM	S3_PM	G1_PM
1	11,682	0.1895	0.1138	0.1596	0.1213	0.2864	0.1687	0.1181	0.1068
2	11,291	0.1976	0.0780	0.1690	0.1392	0.1976	0.2053	0.1585	0.1260
3	12,481	0.0453	0.0610	0.0300	0.0555	0.1799	0.0556	0.0889	0.0596
4	11,518	0.1630	0.1414	0.1233	0.0735	0.4300	0.1210	0.1452	0.0847
5	10,366	0.2971	0.1220	0.3291	0.2140	0.4262	0.3460	0.2130	0.1873
6	11,631	0.0845	0.1240	0.0714	0.1315	0.3211	0.0671	0.1687	0.1107
7	11,164	0.1274	0.1370	0.0666	0.1215	0.4291	0.1161	0.2747	0.1287
8	10,678	0.2110	0.0583	0.0330	0.0837	0.2110	0.0661	0.0976	0.0953
9	9914	0.1817	0.1251	0.0444	0.1377	0.3163	0.0612	0.1573	0.1118
10	10,055	0.3219	0.1383	0.1578	0.1529	0.3658	0.1654	0.1557	0.1965
11	11,139	0.1823	0.0889	0.0929	0.1049	0.2345	0.0980	0.1585	0.0961
12	11,766	0.0792	0.0575	0.0023	0.0654	0.2465	0.0542	0.1180	0.0657
13	9652	0.1311	0.1077	0.0637	0.1095	0.2968	0.1111	0.1274	0.1239
14	10,208	0.2671	0.1444	0.3354	0.1356	0.3362	0.2464	0.2088	0.1171
15	11,422	0.2270	0.0662	0.1199	0.0479	0.4071	0.1255	0.1660	0.0672
16	9118	0.2937	0.0521	0.1067	0.1065	0.2937	0.1278	0.1402	0.0946
17	12,221	0.1689	0.0320	0.0526	0.1003	0.3393	0.1328	0.1416	0.1080
18	12,035	0.3103	0.1450	0.1452	0.1431	0.4162	0.1973	0.2817	0.1465
19	9459	0.1577	0.0769	0.1719	0.1146	0.3630	0.1430	0.0988	0.0763
20	10,900	0.0739	0.0630	0.0441	0.0983	0.2851	0.0672	0.1194	0.0878
21	9857	0.2283	0.1505	0.0476	0.1650	0.2283	0.1590	0.1840	0.1619
22	10,800	0.2477	0.0735	0.1006	0.1056	0.3107	0.1219	0.1501	0.1211
23	11,933	0.2151	0.1228	0.2525	0.0595	0.3551	0.2439	0.1622	0.0474
24	12,042	0.1771	0.0456	0.0634	0.0669	0.3317	0.0490	0.2304	0.0874
25	10,513	0.2560	0.1804	0.0638	0.1862	0.2560	0.1368	0.1796	0.1728
26	12,083	0.2962	0.1114	0.1107	0.1074	0.2962	0.2216	0.2660	0.1011
27	10,861	0.2131	0.0914	0.1711	0.1423	0.2131	0.1260	0.1999	0.1143
28	10,122	0.2780	0.1307	0.1257	0.1146	0.3262	0.1359	0.2043	0.1161
29	13,489	0.1005	0.0904	0.0426	0.1145	0.2902	0.0723	0.1713	0.1162
30	11,760	0.1026	0.0892	0.1051	0.0543	0.3519	0.1168	0.1021	0.0784

Case	Model	G2_PM	G3_PM	G4_PM	FAP_PM	NEH_PM	PPE_PM	LSC_PM
1	11,682	0.1608	0.1608	0.1068	0.0057	0.0223	0.0344	0.0067
2	11,291	0.1438	0.1438	0.1260	0.0099	0.0395	0.0500	0.0112
3	12,481	0.0246	0.0246	0.0596	0.0080	0.0103	0.0208	0.0000
4	11,518	0.1191	0.1191	0.0847	0.0317	0.0703	0.0339	0.0084
5	10,366	0.3405	0.3405	0.1873	0.0141	0.0958	0.0396	0.0213
6	11,631	0.0714	0.0714	0.1107	0.0491	0.0156	0.0349	0.0028
7	11,164	0.0664	0.0664	0.1287	0.0583	0.0202	0.0539	0.0161
8	10,678	0.0330	0.0330	0.0953	0.0332	0.0507	0.0213	0.0000
9	9914	0.0462	0.0462	0.1118	0.0166	0.0232	0.0400	0.0103
10	10,055	0.1636	0.1636	0.1965	0.0513	0.0560	0.0487	0.0180

6.3 Comparison constructive heuristics in small-scale problems

For small-scale instances, the improved constructive heuristics compare with the optimal solutions obtained through the implementation of the MILP model in IBM CPLEX. The new generated benchmark instances with 20 products and 2 fabrication machines are employed here, and hence a total of 30 small-scale instances are solved by the MILP model and constructive heuristics. The comparison results are presented in Table 3. In this table, the model reports the optimal objective values in all small-scale instances; while the improved constructive heuristics report the deviation ratio (DR) which can be calculated by Eq. (33).

$$DR_{heuristic} = \frac{Obj_{heuristic} - Obj_{optimal}}{Obj_{optimal}} \times 100\% \tag{33}$$

where $Obj_{heuristic}$ and $Obj_{optimal}$ mean the objective values obtained by the heuristic and IBM CPLEX respectively.

The results from Table 3 demonstrate that all constructive heuristics adopting the latest PM decision strategy can obtain results less far from the optimal solutions in small-scale instances. Specifically, the proposed LSC_PM obtains the minimum average deviation ratio value 0.0122 resulting in the best results in the quality of solutions. Apart from the LSC_PM heuristic, FAP_AM, PPE_PM and NEH_PM

Table 3 (continued)

Case	Model	G2_PM	G3_PM	G4_PM	FAP_PM	NEH_PM	PPE_PM	LSC_PM
11	11,139	0.0929	0.0929	0.0961	0.0180	0.0563	0.0322	0.0084
12	11,766	0.0023	0.0023	0.0657	0.0104	0.0103	0.0099	0.0012
13	9652	0.0637	0.0637	0.1239	0.0127	0.0367	0.0502	0.0159
14	10,208	0.3280	0.3280	0.1171	0.0206	0.0801	0.0623	0.0168
15	11,422	0.1211	0.1211	0.0672	0.0351	0.0722	0.0381	0.0210
16	9118	0.1067	0.1067	0.0946	0.0197	0.0248	0.0430	0.0160
17	12,221	0.0526	0.0526	0.1080	0.0261	0.0646	0.0366	0.0214
18	12,035	0.1452	0.1452	0.1465	0.0307	0.0366	0.0671	0.0161
19	9459	0.1729	0.1729	0.0763	0.0126	0.0429	0.0299	0.0173
20	10,900	0.0441	0.0441	0.0878	0.0108	0.0092	0.0116	0.0112
21	9857	0.0476	0.0476	0.1619	0.0584	0.0376	0.0486	0.0018
22	10,800	0.1058	0.1058	0.1211	0.0272	0.0612	0.0617	0.0148
23	11,933	0.2525	0.2525	0.0474	0.0285	0.0620	0.0576	0.0085
24	12,042	0.0591	0.0591	0.0874	0.0386	0.0377	0.0500	0.0021
25	10,513	0.0739	0.0739	0.1728	0.0448	0.0289	0.0653	0.0107
26	12,083	0.1117	0.1117	0.1011	0.0326	0.0771	0.0681	0.0252
27	10,861	0.1711	0.1711	0.1143	0.0153	0.0528	0.0733	0.0190
28	10,122	0.1257	0.1257	0.1161	0.0446	0.0452	0.0730	0.0369
29	13,489	0.0423	0.0423	0.1162	0.0331	0.0152	0.0143	0.0046
30	11,760	0.1051	0.1051	0.0784	0.0120	0.0329	0.0273	0.0028

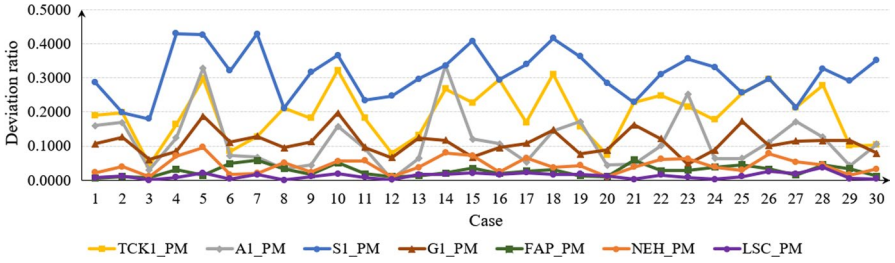

Fig. 11 Deviation ratio of partial constructive heuristics

tend to generate similar results and rank first (their average deviation ratio values are about 0.03). Next, TCK2_PM, A1_PM, A2_PM, G1_PM, G2_PM, G3_PM and G4_PM rank second; S1_PM whose average deviation ratio is 0.3114 obtains the worst results. Heuristic S2_PM follows G2_PM and generates closely results in comparison with TCK1_PM and S3_PM. Besides, Fig. 11 depicts the deviation ratio of partial constructive heuristics in all small-scale instances. This figure further indicates that in most cases, heuristic LSC_PM almost obtains the optimal solutions as the same as those of the model, and its maximum deviation ratio value of small-scale instances does not exceed 0.04. This result suggests the best performance of LSC_PM in small-scale instances in comparison with all the improved constructive heuristics.

6.4 Comparison constructive heuristics in large-scale problems

This section further investigates the performance of all improved constructive heuristics in large-scale instances. The new generated 720 benchmark instances are organized into 24 groups with 30 instances according to the number of products and machines.

Table 4 reports the APRD values of the performance of constructive heuristics. From this table, it can be founded that heuristic LSC_PM can obtain the best solutions in most instances, and it has the minimum mean of ARPD value. This observation suggests that LSC_PM performs best in terms of effectiveness. Following LSC_PM, FAP_PM whose mean of APRD value is 2.23 obtains the second-best results, and PPE_PM whose mean of APRD value is 2.69 obtains the third-best results. As for A1_PM and A2_PM, the former tends to generate closer results to the best solutions. Regarding S1_PM-S3_PM, S2_PM ranks first, following by S3_PM and S1_PM successively. For G1_PM-G4_PM, G1_PM has the same performance as G4_PM, and G2_PM has the same performance as G3_PM. However, G2_PM and G3_PM perform better than G1_PM and G4_PM. In sum, in terms of effectiveness, these heuristics can be divided into six levels: LSC_PM at the highest level; FAP_PM and PPE_PM at the second level; NEH_PM at the third level; TCK2_PM, A1_PM, G2_PM and G3_PM at the fourth level; A2_PM, S2_PM, G1_PM and G4_PM at the fifth level; TCK1_PM and S3_PM and S1_PM at sixth level.

Table 4 ARPD values (30 data per average) of the performance of the heuristics

n	*m*	TCK1_PM	TCK2_PM	A1_PM	A2_PM	S1_PM	S2_PM	S3_PM	G1_PM
20	2	18.04	8.82	10.07	9.99	29.64	12.23	15.30	9.77
	4	15.68	9.25	8.38	10.93	26.10	11.79	18.25	10.10
	6	12.81	8.92	5.23	10.02	21.58	11.60	17.50	8.81
	8	11.79	10.05	5.08	9.33	20.79	12.88	19.13	8.87
40	2	24.49	9.50	17.97	12.85	35.78	18.42	16.07	14.53
	4	19.17	10.32	10.32	13.18	27.71	14.30	20.05	12.80
	6	16.13	9.93	7.64	12.25	23.10	13.72	19.54	10.72
	8	14.80	9.20	6.43	11.44	22.01	12.69	19.93	10.72
60	2	26.35	9.45	19.13	13.60	32.46	19.59	15.69	14.25
	4	19.03	9.83	9.54	13.79	28.62	13.62	20.62	13.14
	6	16.64	9.44	7.13	12.12	25.06	13.73	21.26	11.35
	8	15.45	9.61	8.25	13.28	23.99	13.15	21.77	12.98
80	2	26.09	8.48	21.64	12.33	33.83	21.65	15.58	14.32
	4	20.81	10.19	11.86	14.94	29.04	15.22	21.70	14.54
	6	17.10	9.37	10.23	14.25	26.41	14.59	22.25	13.99
	8	15.96	9.08	5.92	12.54	24.47	13.46	22.09	12.52
100	2	23.55	7.76	15.19	11.56	33.89	16.02	15.26	13.03
	4	20.42	9.12	13.34	14.44	29.92	15.15	21.27	14.42
	6	18.60	9.32	10.60	14.44	26.29	14.56	22.71	14.27
	8	15.51	9.01	6.39	13.00	23.73	13.26	21.19	12.51
120	2	23.74	7.56	15.78	12.11	35.11	16.27	16.13	14.25
	4	20.76	8.65	13.27	15.99	29.21	15.12	21.52	15.69
	6	18.04	8.68	10.71	14.41	25.88	13.45	21.99	14.56
	8	15.74	7.80	7.58	13.32	24.12	12.38	21.13	12.92
Mean		18.61	9.14	10.74	12.75	27.45	14.54	19.50	12.71

n	*m*	G2_PM	G3_PM	G4_PM	FAP_PM	NEH_PM	PPE_PM	LSC_PM
20	2	10.04	10.04	9.77	1.54	3.11	3.14	0.08
	4	8.18	8.18	10.10	2.06	4.33	3.65	0.11
	6	4.64	4.64	8.81	2.08	5.14	3.36	0.07
	8	5.13	5.13	8.87	2.27	4.59	3.13	0.09
40	2	17.86	17.86	14.53	1.64	5.96	3.33	0.10
	4	9.86	9.86	12.80	1.97	5.99	3.42	0.10
	6	6.67	6.67	10.72	2.35	6.26	3.01	0.00
	8	5.62	5.62	10.72	1.78	5.90	3.19	0.01
60	2	19.01	19.01	14.25	1.64	5.99	2.62	0.11
	4	8.93	8.93	13.14	2.14	6.32	2.70	0.02
	6	6.05	6.05	11.35	2.21	6.31	3.02	0.00
	8	7.80	7.80	12.98	2.37	6.36	2.94	0.00
80	2	21.64	21.64	14.32	1.32	6.48	2.40	0.15
	4	11.69	11.69	14.54	2.69	7.46	2.57	0.00
	6	10.13	10.13	13.99	2.25	7.13	2.45	0.00
	8	5.38	5.38	12.52	2.41	7.32	2.67	0.00

Table 4 (continued)

n	m	G2_PM	G3_PM	G4_PM	FAP_PM	NEH_PM	PPE_PM	LSC_PM
100	2	15.22	15.22	13.03	2.42	7.61	1.69	0.00
	4	13.14	13.14	14.42	2.49	7.56	2.43	0.00
	6	10.64	10.64	14.27	2.50	7.64	2.20	0.00
	8	5.95	5.95	12.51	2.49	7.18	2.40	0.00
120	2	15.74	15.74	14.25	3.12	7.86	1.31	0.00
	4	12.94	12.94	15.69	2.75	7.94	2.12	0.00
	6	10.11	10.11	14.56	2.46	7.38	2.34	0.02
	8	7.31	7.31	12.92	2.63	7.27	2.39	0.00
Mean		10.40	10.40	12.71	2.23	6.46	2.69	0.04

The efficiency of 14 constructive heuristics is also measured by calculating their ACPU. Table 5 reports the ACPU values of the performance of the improved constructive heuristics. It can be observed that although LSC_PM has the best performance, it takes the most time; while S1_PM which performs worst takes the least time. Hence, the selection of LSC_PM or S1_PM is subject to the time limitation or objective function that the managers more focus on.

Besides, statistical analysis is of great necessity to further confirm the statistically significant difference. After conducting the normality test and equal variance test, a multiple ANOVA test is carried out where RPD and heuristic types are regarded as the response variable and controlled factor respectively. The ANOVA results for the different heuristic are shown in Table 6. It can be observed from this table that the *P*-value of heuristic types is less than 0.001 with a total of 720 instances. This result demonstrates that the heuristic type has a statistically significant effect on the performance of the integration optimization of two-stage assembly flow shop scheduling and PM. Further, Fig. 12 depicts the means plots of *RPD* with Tukey's Honest Significant Difference (HSD) 95% confidence intervals for all improved constructive heuristics. This figure suggests that the differences between these heuristics are clearly significant: LSC_PM has the best results in the quality of solutions, following by FAP_PM, PPE_PM, NEH_PM, TCK2_PM, G2_PM, G3_PM and A1_PM; whereas S1_PM, S3_PM and TCK1_PM have the first, second and third worst performances.

6.5 Comparison meta-heuristics in all problems

In this section, we aim to measure the performance of the proposed meta-heuristics. The new generated 720 benchmark instances are organized into 24 groups with 30 instances according to the number of products and machines. The proposed 7 meta-heuristics are compared with five existing meta-heuristics to test their performance. the compared meta-heuristics include hybrid tabu search (HTS, (Al-Anzi and Allahverdi 2006)), complete immunoglobulin-based artificial immune system algorithm (CIAIS, (Chung and Chen 2019)), genetic

Table 5 ACPU values (30 data per average) of the performance of the heuristics

n	*m*	TCK1_PM	TCK2_PM	A1_PM	A2_PM	S1_PM	S2_PM	S3_PM	G1_PM
20	2	0.53	0.80	0.67	1.87	0.27	0.13	0.27	19.60
	4	1.07	1.33	1.43	1.60	0.13	0.00	0.13	16.03
	6	3.07	1.33	2.27	1.73	0.40	0.27	0.27	17.20
	8	4.40	0.53	2.00	1.73	0.00	0.13	0.27	15.50
40	2	1.20	0.93	3.73	4.00	0.13	1.20	0.13	73.33
	4	3.07	2.13	4.37	5.07	0.53	0.13	0.40	87.47
	6	5.37	2.13	4.93	4.77	0.40	0.27	0.27	83.60
	8	7.03	2.60	4.53	5.27	0.13	0.53	0.13	70.73
60	2	1.67	1.47	6.47	6.73	0.80	0.23	0.67	179.23
	4	4.13	1.87	7.63	8.17	0.67	0.00	0.53	174.97
	6	7.00	3.13	9.23	9.33	0.27	0.50	0.77	175.60
	8	10.23	3.30	9.67	9.87	0.53	0.53	0.77	175.63
80	2	2.53	2.90	10.37	10.90	0.23	0.63	0.23	330.93
	4	5.57	2.67	12.10	13.53	0.77	1.07	0.53	339.93
	6	8.57	4.20	14.33	14.77	0.13	0.37	0.80	348.37
	8	14.53	5.23	15.20	15.30	0.57	0.93	0.53	351.30
100	2	3.40	3.87	13.87	15.57	0.67	0.93	0.53	550.77
	4	7.03	3.93	17.60	18.13	0.67	0.67	0.93	563.27
	6	12.00	5.23	20.03	20.10	0.80	0.67	1.07	572.17
	8	17.93	5.60	22.43	22.90	0.53	0.67	1.33	583.80
120	2	3.83	3.33	19.77	21.33	0.67	0.87	0.67	946.17
	4	8.10	5.87	24.13	25.37	0.93	0.40	0.40	919.83
	6	13.70	5.97	27.00	28.90	0.13	1.20	0.27	931.83
	8	22.67	5.80	32.83	33.87	1.43	0.40	0.13	960.77
Mean		7.03	3.17	11.94	12.53	0.49	0.53	0.50	353.67

n	*m*	G2_PM	G3_PM	G4_PM	FAP_PM	NEH_PM	PPE_PM	LSC_PM
20	2	20.70	20.53	20.27	25.87	14.90	1.73	29.57
	4	15.70	14.53	15.33	20.00	14.53	2.97	35.47
	6	16.53	17.47	17.60	22.67	13.73	3.60	38.53
	8	16.63	15.80	16.47	20.80	13.60	3.20	41.77
40	2	76.80	75.57	72.80	91.97	54.50	12.83	386.67
	4	89.67	86.17	85.93	120.87	57.63	15.97	473.30
	6	87.73	83.83	81.67	109.93	53.93	16.03	542.50
	8	73.90	71.53	72.50	92.50	59.93	16.90	533.50
60	2	179.20	178.43	175.23	234.37	132.00	31.10	1702.17
	4	180.00	180.37	177.67	236.87	137.37	35.37	2173.37
	6	174.17	173.93	172.80	231.60	140.70	44.37	2574.17
	8	178.50	178.70	174.40	236.50	137.57	44.63	2851.50
80	2	331.40	328.07	329.53	459.03	256.10	62.13	5629.13
	4	337.97	336.87	336.43	465.67	272.10	83.70	6623.47
	6	346.60	347.83	352.07	476.97	274.83	97.17	8193.57
	8	355.80	353.13	352.80	482.00	284.80	112.13	9818.90

Table 5 (continued)

n	*m*	G2_PM	G3_PM	G4_PM	FAP_PM	NEH_PM	PPE_PM	LSC_PM
100	2	543.83	542.40	545.90	780.60	411.27	106.20	15,689.43
	4	570.50	566.80	568.93	800.87	442.47	145.47	17,990.83
	6	587.30	580.17	577.20	814.37	451.93	181.20	19,917.57
	8	593.80	591.87	590.57	831.83	485.20	215.47	26,826.80
120	2	941.10	941.00	943.57	1358.10	626.43	168.73	31,138.87
	4	919.67	919.93	929.93	1345.67	675.97	249.93	41,088.70
	6	927.60	933.53	938.23	1341.80	716.47	274.73	58,093.27
	8	954.73	951.70	967.67	1369.87	802.80	376.33	89,272.77
Mean		354.99	353.76	354.81	498.78	272.12	95.91	14,236.08

Table 6 ANOVA results for the heuristic types

Sources	df	Type III sum of squares	Mean square	*F*-ratio	*P*-value
heuristic types	14	512,075	36,576.8	1925.51	< 0.001
Error	10,785	204,871	19.0		
Total	10,799	716,945			

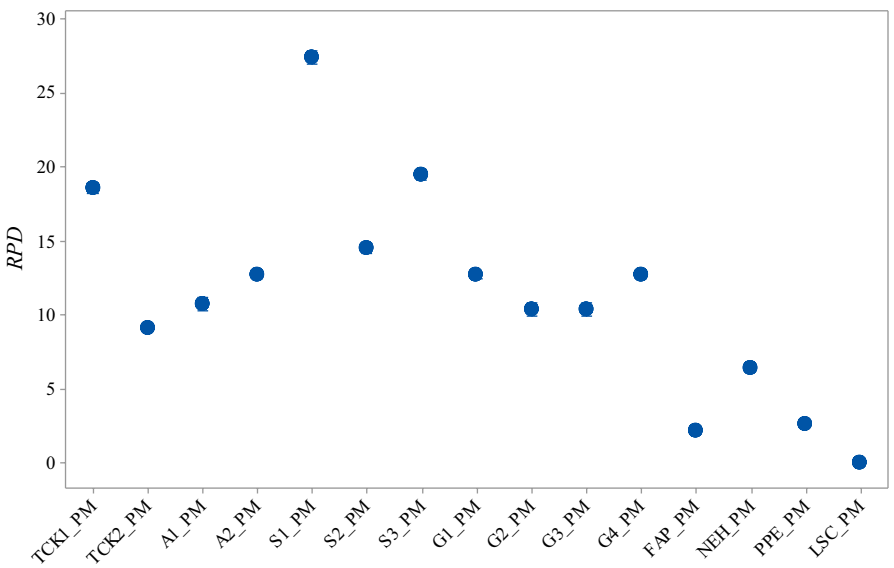

Fig. 12 Means plots of *RPD* with Tukey's Honest Significant Difference (HSD) 95% confidence intervals for all improved constructive heuristics

Table 7 ARPD values (30 data per average) of the performance of the meta-heuristics under CPU time limits of $n \times m \times 5$

n	m	HTS	CIAIS	GA	CSA	NSDE	QHH
20	2	8.94	2.14	15.93	3.34	5.23	0.96
	4	9.72	1.85	13.56	2.80	4.96	0.99
	6	9.89	1.39	11.33	2.40	3.05	1.07
	8	10.65	1.09	10.22	2.05	2.84	0.95
40	2	13.85	11.60	20.69	5.89	11.03	1.64
	4	14.61	7.21	17.52	5.12	7.52	1.61
	6	13.79	4.68	14.36	4.36	5.37	1.66
	8	13.27	3.82	13.41	4.25	4.65	1.38
60	2	14.78	14.87	21.93	6.60	10.78	1.38
	4	13.87	9.16	17.47	5.56	7.64	1.53
	6	13.80	8.08	15.09	4.85	5.95	1.49
	8	13.36	6.14	14.04	4.83	5.28	1.12
80	2	15.10	16.70	22.35	6.97	12.33	1.11
	4	14.75	10.40	17.49	5.71	8.65	1.47
	6	14.23	9.28	15.59	5.17	6.72	1.30
	8	13.25	7.07	14.24	4.76	5.37	0.87
100	2	13.17	17.69	22.02	6.85	12.92	1.85
	4	14.68	14.66	18.34	6.25	8.93	1.60
	6	14.13	10.55	16.06	5.65	7.34	1.58
	8	12.84	9.31	14.11	4.91	5.92	1.68
120	2	14.03	18.66	22.28	7.25	14.06	2.70
	4	14.63	15.21	18.37	6.68	9.85	2.18
	6	13.07	10.86	15.82	5.66	7.26	1.80
	8	11.79	10.03	14.43	5.15	6.15	1.82
Mean		13.18	9.27	16.53	5.13	7.49	1.49
n	m	QACS_LS1	QACS_LS2	QACS_LS3	ILS1_Per2	ILS2_Per1	ILS3_Per1
20	2	0.83	1.56	1.19	0.17	0.80	0.78
	4	1.26	2.13	1.74	0.14	0.90	0.91
	6	1.16	2.08	1.47	0.12	0.83	0.76
	8	1.23	2.14	1.89	0.12	0.80	0.67
40	2	0.83	1.70	1.45	0.38	1.66	1.37
	4	1.06	2.40	1.89	0.39	1.88	1.61
	6	1.11	2.65	1.94	0.34	2.07	1.52
	8	1.25	2.29	1.74	0.35	1.81	1.37
60	2	1.94	1.39	2.05	0.18	1.18	0.83
	4	0.68	2.14	1.28	0.36	1.98	1.20
	6	0.64	2.31	1.53	0.27	1.96	1.29
	8	0.92	2.31	1.72	0.41	1.95	1.40

Table 7 (continued)

n	m	QACS_LS1	QACS_LS2	QACS_LS3	ILS1_Per2	ILS2_Per1	ILS3_Per1
80	2	1.43	1.43	1.43	0.07	0.85	0.70
	4	0.01	2.54	2.56	0.01	2.02	0.92
	6	0.38	1.85	1.20	0.17	1.73	1.12
	8	0.26	1.80	1.31	0.18	1.90	1.07
100	2	2.35	2.35	2.35	0.00	1.70	0.65
	4	2.32	2.32	2.32	0.00	1.96	0.92
	6	1.19	2.03	1.11	0.09	1.92	1.01
	8	0.12	1.79	1.21	0.08	1.95	1.17
120	2	2.92	2.92	2.92	0.10	2.07	0.68
	4	2.53	2.53	2.53	0.01	2.33	0.94
	6	1.66	2.02	2.19	0.00	1.98	1.07
	8	0.90	1.97	1.37	0.03	2.02	1.31
Mean		1.21	2.11	1.77	0.17	1.68	1.05

algorithm (GA, Seidgar et al. 2019), cloud theory-based simulated annealing (CSA, Torabzadeh and Zandieh 2010) and new self-adapted differential evolutionary (NSDE, (Seidgar et al. 2019). For a fair comparison, 12 meta-heuristics are solved under two same stopping criteria with CPU time limits of $n \times m \times 5$ and $n \times m \times 10$ ms. The APRD values of the performance of the meta-heuristics are shown in Tables 7, 8.

From this table, it can be found that all the proposed meta-heuristics (QHH, QACS_LS1, QACS_LS2, QACS_LS3, ILS1_Per2, ILS2_Per1 and ILS3_Per1) outperform the five existing meta-heuristics (HTS, CIAIS, GA, CSA and NSDE) under two stopping criteria. Specifically, ILS1_Per2 has the best performance in all instances, whose ARPD has the smallest values of 0.17 and 0.27 under two stopping criteria. GA and HTS have the worst performance. If using the RPD values under $n \times m \times 10$ to rank these meta-heuristics: ILS1_Per2 ranks first, following by ILS2_Per1, ILS3_Per1, QACS_LS1, QHH, QACS_LS3, QACS_LS2, CSA, NSDE, CIAIS, HTS and GA. To further confirm the statistically significant difference, a statistical analysis is carried out. Figure 13 depicts the Means plots of *RPD* with Tukey's Honest Significant Difference (HSD) 95% confidence intervals for all meta-heuristics under $n \times m \times 10$. As shown in this figure, 12 meta-heuristics can be divided into 4 levels in terms of effectiveness: the proposed 7 meta-heuristics monopolize the first level, which means these meta-heuristics are far superior to other meta-heuristics; CSA, NSDE and CIAIS are at the second level; HTS is at the third level; GA is at the last level, which means it performs worst. Hence we can conclude that the proposed meta-heuristics are significantly better than other existing meta-heuristics where ILS1_Per2 performs best.

Table 8 ARPD values (30 data per average) of the performance of the meta-heuristics under CPU time limits of $n \times m \times 10$

n	m	HTS	CIAIS	GA	CSA	NSDE	QHH
20	2	8.71	1.43	15.67	2.50	5.25	0.74
	4	9.22	1.02	13.27	2.36	3.33	0.85
	6	9.57	0.85	11.20	2.13	2.35	0.91
	8	10.22	0.76	10.07	1.86	1.81	0.91
40	2	13.86	7.85	20.58	5.43	8.54	1.39
	4	14.70	4.43	17.41	4.89	5.56	1.42
	6	13.85	3.04	14.39	4.18	4.25	1.22
	8	13.31	2.78	13.46	4.13	3.56	1.03
60	2	14.65	10.68	21.79	6.09	8.99	0.97
	4	13.81	7.21	17.46	5.22	6.00	1.13
	6	13.92	5.44	15.19	4.74	4.56	0.71
	8	13.67	4.24	14.39	4.89	4.29	0.76
80	2	15.21	13.19	22.46	6.75	10.66	1.02
	4	15.10	8.89	17.95	5.79	6.69	1.15
	6	14.57	6.84	15.90	5.30	5.08	0.47
	8	13.74	5.74	14.62	5.00	4.34	0.49
100	2	13.38	17.90	22.14	6.74	10.19	1.69
	4	14.99	12.30	18.66	6.28	7.25	1.19
	6	14.70	9.46	16.61	5.98	5.89	0.60
	8	13.41	7.35	14.64	5.25	4.78	0.40
120	2	14.25	18.93	22.58	7.14	11.36	2.15
	4	14.91	13.09	18.62	6.71	7.78	1.39
	6	13.54	9.91	16.33	5.99	5.99	0.63
	8	12.39	8.96	15.04	5.54	5.04	0.46
Mean		13.32	7.60	16.69	5.04	5.98	0.99
n	m	QACS_LS1	QACS_LS2	QACS_LS3	ILS1_Per2	ILS2_Per1	ILS3_Per1
20	2	0.84	1.57	1.20	0.12	0.56	0.55
	4	1.26	2.13	1.73	0.09	0.70	0.67
	6	1.16	2.09	1.50	0.08	0.55	0.54
	8	1.23	2.15	1.90	0.08	0.56	0.48
40	2	0.88	1.75	1.50	0.42	0.69	0.85
	4	1.16	2.50	1.99	0.41	1.02	1.06
	6	1.20	2.74	2.03	0.24	0.67	0.75
	8	1.33	2.37	1.82	0.23	0.88	0.89
60	2	0.19	1.23	0.74	0.38	0.38	0.39
	4	0.65	2.10	1.25	0.28	0.33	0.38
	6	0.76	2.44	1.65	0.25	0.34	0.38
	8	1.19	2.60	2.00	0.36	0.63	0.64

Table 8 (continued)

n	*m*	QACS_LS1	QACS_LS2	QACS_LS3	ILS1_Per2	ILS2_Per1	ILS3_Per1
80	2	0.19	1.00	0.79	0.36	0.36	0.36
	4	0.32	2.57	1.32	0.19	0.19	0.19
	6	0.58	2.15	1.51	0.20	0.20	0.20
	8	0.70	2.25	1.75	0.33	0.33	0.33
100	2	0.19	1.90	0.87	0.11	0.11	0.11
	4	0.28	2.39	1.21	0.12	0.12	0.12
	6	0.58	2.54	1.59	0.28	0.28	0.28
	8	0.62	2.30	1.72	0.22	0.22	0.22
120	2	3.12	3.12	3.12	0.00	0.00	0.00
	4	0.25	2.64	1.18	0.03	0.03	0.03
	6	0.43	2.45	1.50	0.20	0.20	0.20
	8	0.54	2.52	1.91	0.12	0.12	0.12
Mean		0.82	2.23	1.57	0.21	0.39	0.41

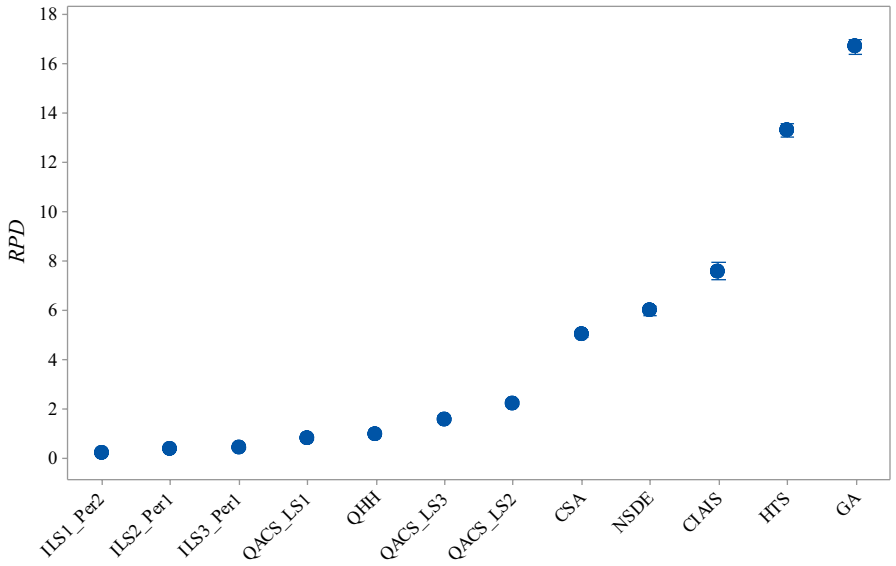

Fig. 13 Means plots of *RPD* with Tukey's Honest Significant Difference (HSD) 95% confidence intervals for all meta-heuristics under $n \times m \times 10$

7 Conclusions and future work

In real production, machines are inevitably subject to some unavailable periods due to unexpected failure or preventive maintenance with time elapsing. This situation will no doubt lead to production stoppages. Hence, to ensure the machine reliability

and production continuity, flexible preventive maintenance activities are first incorporated as a constraint to two-stage assembly flow shop scheduling. In this new problem, each machine is given a new feature maintenance level whose initial value is determined according to the optimal maintenance interval of Weibull probability distribution. Flexible PM activities can be executed at any time to assure that the maintenance levels of all machines are not less than 0 at any time. We aim to develop a product sequence and PM activity execution time points with the minimum total completion time and maintenance time. A MILP model, a latest PM decision strategy, 15 improved constructive heuristics and 7 meta-heuristics are hence developed to solve this new problem. Finally, three types of experiments are conducted to demonstrate the significance of integration optimization of scheduling and PM and evaluate the performance of 15 constructive heuristics and 7 meta-heuristics. The comprehensive statistical and computational results lead clearly to three conclusions.

1. It is significantly crucial to incorporate flexible PM to two-stage assembly flow shop scheduling. An optimal or near-optimal decision on product sequence and PM execution time points can greatly reduce the completion time and maintenance costs;
2. The proposed latest PM decision strategy helps all constructive heuristics in obtaining results with a slight difference to the optimal solutions in 720 benchmark instances;
3. Although the proposed LSC_PM takes the most time, it performs best in 720 benchmark instances regarding effectiveness. As for the meta-heuristics, the proposed 7 meta-heuristics outperform the other 5 existing meta-heuristics, and the proposed ILS1_Per2 has the best performance under the same stopping criteria.

Based on this work, future research will study bi-objective of total completion time or maintenance costs and design a Pareto meta-heuristic to tackle this new problem. Another suggestion is to consider the different release maintenance levels. That is, in the beginning, all the machines are not at the optimal maintenance levels. Meanwhile, several different maintenance activities could be involved.

Appendices A: The flowchart of TCK1_PM

Appendices A The flowchart of TCK1_PM

Heuristic: TCK1_PM

For $s \leftarrow 1$ to m+1 do
  $J_{un} = \{1, \ldots, n\}$;
  For $i \leftarrow 1$ to n do //construct product sequence
    $\pi_{si} \leftarrow \arg\min_{j \in J_{un}} t_{js}(or\ p_j)$; //p_j is used in assembly stage
    $J_{un} = J_{un} - \pi_{si}$;
  For $k \leftarrow 1$ to m+1 do //determine objective functions
    $ML_{0k}^{a} \leftarrow ml_k^0$; $C_{0k} \leftarrow 0$;
    For $i \leftarrow 1$ to n do
      If $ML_{i-1,k}^{a} - \mu_k \cdot t_{\pi_{si},k}(or\ p_{\pi_{si}}) < 0$ then //$p_{\pi_{si}}$ is used in assembly stage
        $Y_{ik} \leftarrow 1$;
        $ML_{ik}^{a} \leftarrow ml_k^0 - \mu_k \cdot t_{\pi_{si},k}(or\ p_{\pi_{si}})$; //$p_{\pi_{si}}$ is used in assembly stage
        $C_{ik} \leftarrow C_{i-1,k} + t_{\pi_{si},k} + dpm_k$; //used in fabrication machines
        $C_{i,m+1} \leftarrow min\{C_{ik}, C_{i-1,m+1} + dpm_k\} + p_{\pi_{si}}$; //used in assembly machine
      Else:
        $Y_{ik} \leftarrow 0$;
        $ML_{ik}^{a} \leftarrow ML_{i-1,k}^{a} - \mu_k \cdot t_{\pi_{si},k}(or\ p_{\pi_{si}})$; //$p_{\pi_{si}}$ is used in assembly stage
        $C_{ik} \leftarrow C_{i-1,k} + t_{\pi_{si},k}$; //used in fabrication machines
        $C_{i,m+1} \leftarrow min\{C_{ik}, C_{i-1,m+1}\} + p_{\pi_{si}}$; //used in assembly machine
  $Obj_s \leftarrow \sum_{i=1}^{n} C_{i,m+1} + \sum_{i=1}^{n} \sum_{k=1}^{m+1} Y_{ik} \cdot dpm_k$;//calculate objective value
$\Pi_{selected} \leftarrow \arg\min_{\Pi_s} Obj_s$;

Note that J_{un} is a set of unassigned products.

Appendix B: The flowcharts of S1_PM, S2_PM and S3_PM

Appendix B The flowcharts of S1_PM, S2_PM and S3_PM

Heuristics: S1_PM, S2_PM and S3_PM

$J_{un} = \{1, \ldots, n\}$;
For $i \leftarrow 1$ to n do //construct product sequence
    $\pi_i \leftarrow \arg\min_{j \in J_{un}} p_j$; //used in S1_PM
    $\pi_i \leftarrow \arg\min_{j \in J_{un}} \left\{\max_{1 \le k \le m} t_{jk}\right\}$; //used in S2_PM
    $\pi_i \leftarrow \arg\min_{j \in J_{un}} \left\{\max_{1 \le k \le m} t_{jk} + p_j\right\}$; //used in S3_PM
    $J_{un} = J_{un} - \pi_i$;
For $k \leftarrow 1$ to m+1 do //determine objective functions
    $ML^a_{0k} \leftarrow ml^0_k$; $C_{0k} \leftarrow 0$;
    For $i \leftarrow 1$ to n do
        If $ML^a_{i-1,k} - \mu_k \cdot t_{\pi_{si},k}(or\ p_{\pi_{si}}) < 0$ then //$p_{\pi_{si}}$ is used in assembly stage
            $Y_{ik} \leftarrow 1$;
            $ML^a_{ik} \leftarrow ml^0_k - \mu_k \cdot t_{\pi_{si},k}(or\ p_{\pi_{si}})$; //$p_{\pi_{si}}$ is used in assembly stage
            $C_{ik} \leftarrow C_{i-1,k} + t_{\pi_{si},k} + dpm_k$; //used in fabrication machines
            $C_{i,m+1} \leftarrow min\{C_{ik}, C_{i-1,m+1} + dpm_k\} + p_{\pi_{si}}$; //used in assembly machine
        Else:
            $Y_{ik} \leftarrow 0$;
            $ML^a_{ik} \leftarrow ML^a_{i-1,k} - \mu_k \cdot t_{\pi_{si},k}(or\ p_{\pi_{si}})$; //$p_{\pi_{si}}$ is used in assembly stage
            $C_{ik} \leftarrow C_{i-1,k} + t_{\pi_{si},k}$; //used in fabrication machines
            $C_{i,m+1} \leftarrow min\{C_{ik}, C_{i-1,m+1}\} + p_{\pi_{si}}$; //used in assembly machine
$Obj \leftarrow \sum_{i=1}^{n} C_{i,m+1} + \sum_{i=1}^{n} \sum_{k=1}^{m+1} Y_{ik} \cdot dpm_k$;//calculate objective value

Appendix C The flowcharts of A1_PM and A2_PM

Appendix C The flowcharts of A1_PM and A2_PM

Heuristics: A1_PM and A2_PM

$J_{un} = \{1, \ldots, n\}$; $J_{as} = \emptyset$;
For $i \leftarrow 1$ to n do //construct product sequence
  $\pi_i \leftarrow \arg\min_{j \in J_{un}} \left\{ \max_{1 \le k \le m} \left(\sum_{r \in J_{as}} t_{rk} + t_{jk} \right) \right\}$; //used in A1_PM
  $\pi_i \leftarrow \arg\min_{j \in J_{un}} \left\{ \max_{1 \le k \le m} \left(\sum_{r \in J_{as}} t_{rk} + t_{jk} \right) + p_j \right\}$; //used in A2_PM
  $J_{un} = J_{un} - \pi_i$; $J_{as} = J_{as} + \pi_i$;
For $k \leftarrow 1$ to m+1 do //determine objective functions
  $ML_{0k}^{a} \leftarrow ml_k^0$; $C_{0k} \leftarrow 0$;
  For $i \leftarrow 1$ to n do
    If $ML_{i-1,k}^{a} - \mu_k \cdot t_{\pi_{si},k} (or\ p_{\pi_{si}}) < 0$ then //$p_{\pi_{si}}$ is used in assembly stage
      $Y_{ik} \leftarrow 1$;
      $ML_{ik}^{a} \leftarrow ml_k^0 - \mu_k \cdot t_{\pi_{si},k} (or\ p_{\pi_{si}})$; //$p_{\pi_{si}}$ is used in assembly stage
      $C_{ik} \leftarrow C_{i-1,k} + t_{\pi_{si},k} + dpm_k$; //used in fabrication machines
      $C_{i,m+1} \leftarrow min\{C_{ik}, C_{i-1,m+1} + dpm_k\} + p_{\pi_{si}}$; //used in assembly machine
    Else:
      $Y_{ik} \leftarrow 0$;
      $ML_{ik}^{a} \leftarrow ML_{i-1,k}^{a} - \mu_k \cdot t_{\pi_{si},k} (or\ p_{\pi_{si}})$; //$p_{\pi_{si}}$ is used in assembly stage
      $C_{ik} \leftarrow C_{i-1,k} + t_{\pi_{si},k}$; //used in fabrication machines
      $C_{i,m+1} \leftarrow min\{C_{ik}, C_{i-1,m+1}\} + p_{\pi_{si}}$; //used in assembly machine
$Obj \leftarrow \sum_{i=1}^{n} C_{i,m+1} + \sum_{i=1}^{n} \sum_{k=1}^{m+1} Y_{ik} \cdot dpm_k$;//calculate objective value

Appendix D: The flowcharts of G1_PM, G2_PM, G3_PM and G4_PM

Appendix D The flowcharts of G1_PM, G2_PM, G3_PM and G4_PM

Heuristics: G1_PM, G2_PM, G3_PM and G4_PM

$J_{un} = \{1, \ldots, n\}$; $ML_{0k}^{a} \leftarrow ml_{k}^{0}$; $C_{0k} \leftarrow 0$;
For $k \leftarrow 1$ to m+1 do
  $ML_{0k}^{a} \leftarrow ml_{k}^{0}$; $C_{0k} \leftarrow 0$;
For $i \leftarrow 1$ to n do //construct product sequence
  For $j \in J_{un}$ do
    For $k \leftarrow 1$ to m+1 do //calculate completion time and PM decision
      If $ML_{i-1,k}^{a} - \mu_k \cdot t_{jk}(or\ p_j) < 0$ then //p_j is used in assembly stage
        $A_{ji}[Y_{ik}] \leftarrow 1$;
        $A_{ji}[ML_{ik}^{a}] \leftarrow ml_{k}^{0} - \mu_k \cdot t_{jk}(or\ p_j)$; //$p_j$ is used in assembly stage
        $A_{ji}[C_{ik}] \leftarrow C_{i-1,k} + t_{jk} + dpm_k$; //fabrication machines
        $A_{ji}[C_{i,m+1}] \leftarrow min\{A_{ji}[C_{ik}], C_{i-1,m+1} + dpm_k\} + p_j$; //assembly stage
      Else:
        $A_{ji}[Y_{ik}] \leftarrow 0$;
        $A_{ji}[ML_{ik}^{a}] \leftarrow ML_{i-1,k}^{a} - \mu_k \cdot t_{jk}(or\ p_j)$; //$p_j$ is used in assembly stage
        $A_{ji}[C_{ik}] \leftarrow C_{i-1,k} + t_{jk}$; // fabrication machines
        $A_{ji}[C_{i,m+1}] \leftarrow min\{A_{ji}[C_{ik}], C_{i-1,m+1}\} + p_j$; // assembly machine
  $\pi_i \leftarrow \arg\min_{j \in J_{un}} \{A_{ji}[C_{i,m+1}] - C_{i-1,m+1}\}$; //used in G1_PM
  $\pi_i \leftarrow \arg\min_{j \in J_{un}} \left\{\max_{1 \le k \le m}\left(A_{ji}[C_{ik}]\right) - \max_{1 \le k \le m}\left(C_{i-1,k}\right)\right\}$; //used in G2_PM
  $\pi_i \leftarrow \arg\min_{j \in J_{un}} \left\{\max_{1 \le k \le m}\left(A_{ji}[C_{ik}]\right) - C_{i-1,m+1}\right\}$; //used in G3_PM
  $\pi_i \leftarrow \arg\min_{j \in J_{un}} \left\{A_{ji}[C_{i,m+1}] - \max_{1 \le k \le m}\left(C_{i-1,k}\right)\right\}$; //used in G4_PM
  For $k \leftarrow 1$ to m+1 do
    $Y_{ik} \leftarrow A_{\pi_i,i}[Y_{ik}]$;
    $ML_{ik}^{a} \leftarrow A_{\pi_i,i}[ML_{ik}^{a}]$;
    $C_{ik} \leftarrow A_{\pi_i,i}[C_{ik}]$; //fabrication machines
    $C_{i,m+1} \leftarrow mA_{\pi_i,i}[C_{i,m+1}]$; //assembly machine
$Obj \leftarrow \sum_{i=1}^{n} C_{i,m+1} + \sum_{i=1}^{n} \sum_{k=1}^{m+1} Y_{ik} \cdot dpm_k$;//calculate objective value

Appendix E: The flowchart of FAP_PM

Appendix E The flowchart of FAP_PM

Heuristic: FAP_PM

$J_{un} = \{1, \ldots, n\}$; $ML_{0k}^{a} \leftarrow ml_{k}^{0}$; $C_{0k} \leftarrow 0$;
For $k \leftarrow 1$ to m+1 do
  $ML_{0k}^{a} \leftarrow ml_{k}^{0}$; $C_{0k} \leftarrow 0$;
For $i \leftarrow 1$ to n do //construct product sequence
  For $j \in J_{un}$ do
    For $k \leftarrow 1$ to m+1 do //calculate completion time and PM decision
      If $ML_{i-1,k}^{a} - \mu_k \cdot t_{jk}(or\ p_j) < 0$ then //p_j is used in assembly stage
        $A_{ji}[Y_{ik}] \leftarrow 1$;
        $A_{ji}[ML_{ik}^{a}] \leftarrow ml_{k}^{0} - \mu_k \cdot t_{jk}(or\ p_j)$; //$p_j$ is used in assembly stage
        $A_{ji}[C_{ik}] \leftarrow C_{i-1,k} + t_{jk} + dpm_k$; //fabrication machines
        $A_{ji}[C_{i,m+1}] \leftarrow min\{A_{ji}[C_{ik}], C_{i-1,m+1} + dpm_k\} + p_j$; //assembly stage
      Else:
        $A_{ji}[Y_{ik}] \leftarrow 0$;
        $A_{ji}[ML_{ik}^{a}] \leftarrow ML_{i-1,k}^{a} - \mu_k \cdot t_{jk}(or\ p_j)$; //$p_j$ is used in assembly stage
        $A_{ji}[C_{ik}] \leftarrow C_{i-1,k} + t_{jk}$; // fabrication machines
        $A_{ji}[C_{i,m+1}] \leftarrow min\{A_{ji}[C_{ik}], C_{i-1,m+1}\} + p_j$; // assembly machine
    If $C_{ji}^{1} > C_{2}^{*}$ then //calculate indicator
      $FAP_{ji} = C_{ji}^{1} + \frac{(n-i+1)}{n} \cdot \left(C_{1\blacksquare} + \frac{p_{\blacksquare}}{m}\right)$;
    Else:
      $FAP_{ji} = p_j + \frac{(n-i+1)}{n}\left(C_{1\blacksquare} + \frac{p_{\blacksquare}}{m}\right)$;
  $\pi_i \leftarrow \arg\min_{j \in J_{un}}\{FAP_{ji}\}$;
  For $k \leftarrow 1$ to m+1 do
    $Y_{ik} \leftarrow A_{\pi_i,i}[Y_{ik}]$;
    $ML_{ik}^{a} \leftarrow A_{\pi_i,i}[ML_{ik}^{a}]$;
    $C_{ik} \leftarrow A_{\pi_i,i}[C_{ik}]$; //fabrication machines
    $C_{i,m+1} \leftarrow mA_{\pi_i,i}[C_{i,m+1}]$; //assembly machine
$Obj \leftarrow \sum_{i=1}^{n} C_{i,m+1} + \sum_{i=1}^{n}\sum_{k=1}^{m+1} Y_{ik} \cdot dpm_k$;//calculate objective value

Acknowledgement This work is supported by National Natural Science Foundation of China (No. 51875421).

References

Al-Anzi FS, Allahverdi A (2006) A hybrid Tabu search heuristic for the two-stage assembly scheduling problem. Int J Oper Res 3: 109–119

Al-Anzi FS, Allahverdi A (2007) A self-adaptive differential evolution heuristic for two-stage assembly scheduling problem to minimize maximum lateness with setup times. Eur J Oper Res 182: 80–94
Al-Anzi FS, Allahverdi A (2013) An artificial immune system heuristic for two-stage multi-machine assembly scheduling problem to minimize total completion time. J Manuf Syst 32: 825–830
Allahverdi A, Al-Anzi FS (2009) The two-stage assembly scheduling problem to minimize total completion time with setup times. Comput Oper Res 36:2740–2747
Allahverdi A, Aydilek H (2013) The two stage assembly flowshop scheduling problem to minimize total tardiness. J Intell Manuf 26:225–237
Allahverdi A, Aydilek H, Aydilek A (2016) Two-stage assembly scheduling problem for minimizing total tardiness with setup times. Appl Math Model 40:7796–7815
Aydilek A, Aydilek H, Allahverdi A (2017) Minimising maximum tardiness in assembly flowshops with setup times. Int J Prod Res 55:7541–7565
Baykasoglu A, Dereli T (2008) Two-sided assembly line balancing using an ant-colony-based heuristic. Int J Adv Manuf Technol 36:582–588
Blocher JD, Chhajed D (2007) Minimizing customer order lead-time in a two-stage assembly supply chain. Ann Oper Res 161:25–52
Bock S, Briskorn D, Horbach A (2011) Scheduling flexible maintenance activities subject to job-dependent machine deterioration. J Sched 15:565–578
Burke EK, Bykov Y (2008) A late acceptance strategy in hill-climbing for examination timetabling problems. In: Conference on the practice & theory of automated timetabling
Chung TP, Chen F (2019) A complete immunoglobulin-based artificial immune system algorithm for two-stage assembly flowshop scheduling problem with part splitting and distinct due windows. Int J Prod Res 57:3219–3237
El Khoukhi F, Boukachour J, Alaoui AEH (2017) The "Dual-Ants Colony": a novel hybrid approach for the flexible job shop scheduling problem with preventive maintenance. Comput Ind Eng 106: 236–255.
Fattahi P, Hosseini SMH, Jolai F (2013) A mathematical model and extension algorithm for assembly flexible flow shop scheduling problem. Int J Adv Manuf Technol 65:787–802
Fernandez-Viagas V, Framinan JM (2014) A bounded-search iterated greedy algorithm for the distributed permutation flowshop scheduling problem. Int J Prod Res 53:1111–1123
Framinan JM, Perez-Gonzalez P (2017) The 2-stage assembly flowshop scheduling problem with total completion time: efficient constructive heuristic and metaheuristic. Comput Oper Res 88:237–246
Framinan JM, Perez-Gonzalez P, Fernandez-Viagas V (2019) Deterministic assembly scheduling problems: a review and classification of concurrent-type scheduling models and solution procedures. Eur J Oper Res 273:401–417
Jung S, Woo Y-B, Kim BS (2017) Two-stage assembly scheduling problem for processing products with dynamic component-sizes and a setup time. Comput Ind Eng 104:98–113
Kazemi H, Mazdeh MM, Rostami M (2017) The two stage assembly flow-shop scheduling problem with batching and delivery. Eng Appl Artif Intell 63: 98–107
Komaki GM, Teymourian E, Kayvanfar V (2015) Minimising makespan in the two-stage assembly hybrid flow shop scheduling problem using artificial immune systems. Int J Prod Res 54:963–983
Komaki GM, Sheikh S, Malakooti B (2019) Flow shop scheduling problems with assembly operations: a review and new trends. Int J Prod Res 57:2926–2955
Lee IS (2018) Minimizing total completion time in the assembly scheduling problem. Comput Ind Eng 122:211–218
Lee C-Y, Cheng TCE, Lin BMT (1993) Minimizing the makespan in the 3-machine assembly-type flowshop scheduling problem. Manag Sci 39:616–625
Li J-Q, Pan Q-K (2012) Chemical-reaction optimization for flexible job-shop scheduling problems with maintenance activity. Appl Soft Comput 12:2896–2912
Li J-Q, Pan Q-K, Tasgetiren MF (2014) A discrete artificial bee colony algorithm for the multi-objective flexible job-shop scheduling problem with maintenance activities. Appl Math Model 38:1111–1132
Liao C-J, Lee C-H, Lee H-C (2015) An efficient heuristic for a two-stage assembly scheduling problem with batch setup times to minimize makespan. Comput Ind Eng 88:317–325
Lin WC (2018) Minimizing the Makespan for a two-stage three-machine assembly flow shop problem with the sum-of-processing-time based learning effect, Discrete Dynamics in Nature and Society
Minella G, Ruiz R, Ciavotta M (2011) Restarted Iterated Pareto Greedy algorithm for multi-objective flowshop scheduling problems. Comput Oper Res 38:1521–1533

Miyata HH, Nagano MS, Gupta JND (2019) Integrating preventive maintenance activities to the no-wait flow shop scheduling problem with dependent-sequence setup times and makespan minimization. Comput Ind Eng 135:79–104
Mosheiov G, Sarig A, Strusevich VA, Mosheiff J (2018) Two-machine flow shop and open shop scheduling problems with a single maintenance window. Eur J Oper Res 271:388–400
Naderi B, Zandieh M, Aminnayeri M (2011) Incorporating periodic preventive maintenance into flexible flowshop scheduling problems. Appl Soft Comput 11:2094–2101
Navaei J, Ghomi S, Jolai F, Shiraqai ME, Hidaji H (2013) Two-stage flow-shop scheduling problem with non-identical second stage assembly machines. Int J Adv Manuf Technol 69:2215–2226
Navaei J, Ghomi S, Jolai F, Mozdgir A (2014) Heuristics for an assembly flow-shop with non-identical assembly machines and sequence dependent setup times to minimize sum of holding and delay costs. Comput Oper Res 44:52–65
Potts CN, Sevast'janov SV, Strusevich VA, Van Wassenhove LN, Zwaneveld CM (1995) The two-stage assembly scheduling problem: complexity and approximation. Oper Res 43:346–355
Ruiz R, García-Díaz JC, Maroto C (2007) Considering scheduling and preventive maintenance in the flowshop sequencing problem. Comput Oper Res 34: 3314–3330.
Safari E, Jafar Sadjadi S, Shahanaghi K (2009) Scheduling flowshops with condition-based maintenance constraint to minimize expected makespan. Int J Adv Manuf Technol 46: 757–767
Seidgar H, Kiani M, Abedi M, Fazlollahtabar H (2013) An efficient imperialist competitive algorithm for scheduling in the two-stage assembly flow shop problem. Int J Prod Res 52:1240–1256
Seidgar H, Zandieh M, Fazlollahtabar H, Mahdavi I (2016a) Simulated imperialist competitive algorithm in two-stage assembly flow shop with machine breakdowns and preventive maintenance. Proc Inst Mech Eng Part B-J Eng Manuf 230:934–953
Seidgar H, Zandieh M, Mahdavi I (2016b) Bi-objective optimization for integrating production and preventive maintenance scheduling in two-stage assembly flow shop problem. J Ind Prod Eng 33:404–425
Seidgar H, Fazlollahtabar H, Zandieh M (2019) Scheduling two-stage assembly flow shop with random machines breakdowns: integrated new self-adapted differential evolutionary and simulation approach. Soft Computing
Seif J, Dehghanimohammadabadi M, Yu AJ (2019) Integrated preventive maintenance and flow shop scheduling under uncertainty, Flex Serv Manuf J
Shamsaei F, Van Vyve M (2016) Solving integrated production and condition-based maintenance planning problems by MIP modeling. Flex Serv Manuf J 29:184–202
Sheikhalishahi M, Eskandari N, Mashayekhi A, Azadeh A (2019) Multi-objective open shop scheduling by considering human error and preventive maintenance. Appl Math Model 67:573–587
Sung CS, Kim HA (2008) A two-stage multiple-machine assembly scheduling problem for minimizing sum of completion times. Int J Prod Econ 113:1038–1048
Talens C, Fernandez-Viagas V, Perez-Gonzalez P, Framinan JM (2020) New efficient constructive heuristics for the two-stage multi-machine assembly scheduling problem. Comput Ind Eng 140
Torabzadeh E, Zandieh M (2010) Cloud theory-based simulated annealing approach for scheduling in the two-stage assembly flowshop. Adv Eng Softw 41:1238–1243
Tozkapan A, Kırca Ö, Chung C-S (2003) A branch and bound algorithm to minimize the total weighted flowtime for the two-stage assembly scheduling problem. Comput Oper Res 30:309–320
Wang S, Liu M (2014) Two-stage hybrid flow shop scheduling with preventive maintenance using multi-objective tabu search method. Int J Prod Res 52:1495–1508
Wu CC, Wang DJ, Cheng SR, Chung IH, Lin WC (2018a) A two-stage three-machine assembly scheduling problem with a position-based learning effect. Int J Prod Res 56:3064–3079
Wu CC, Chen JY, Lin WC, Lai KJ, Liu SC, Yu PW (2018b) A two-stage three-machine assembly flow shop scheduling with learning consideration to minimize the flowtime by six hybrids of particle swarm optimization. Swarm Evolutionary Comput 41:97–110
Wu CC, Chen JY, Lin WC, Lai K, Bai D, Lai SY (2019) A two-stage three-machine assembly scheduling flowshop problem with both two-agent and learning phenomenon. Comput Ind Eng 130:485–499
Ye H, Wang X, Liu K (2020) Adaptive preventive maintenance for flow shop scheduling with resumable processing. IEEE Trans Autom Sci Eng 1–8
Yokoyama M (2001) Hybrid flow-shop scheduling with assembly operations. Int J Prod Econ 73:103–116
Yokoyama M (2004) Scheduling for two-stage production system with setup and assembly operations. Comput Oper Res 31:2063–2078

Yokoyama M, Santos DL (2005) Three-stage flow-shop scheduling with assembly operations to minimize the weighted sum of product completion times. Eur J Oper Res 161:754–770
Yu AJ, Seif J (2016) Minimizing tardiness and maintenance costs in flow shop scheduling by a lower-bound-based GA. Comput Ind Eng 97:26–40
Zandieh M, Khatami AR, Rahmati SHA (2017) Flexible job shop scheduling under condition-based maintenance: improved version of imperialist competitive algorithm. Appl Soft Comput 58:449–464
Zhang Y, Zhou Z, Liu J (2010) The production scheduling problem in a multi-page invoice printing system. Comput Oper Res 37:1814–1821
Zhang Z, Tang Q, Ruiz R, Zhang L (2020) Ergonomic risk and cycle time minimization for the U-shaped worker assignment assembly line balancing problem: a multi-objective approach. Comput Oper Res 118
Zhang Z, Tang Q, Chica M (2020) Multi-manned assembly line balancing with time and space constraints: a MILP model and memetic ant colony system. Comput Ind Eng
Zhou X, Lu Z, Xi L (2012) Preventive maintenance optimization for a multi-component system under changing job shop schedule. Reliab Eng Syst Saf 101:14–20

Zikai Zhang was born in Xiangyang, Hubei, China, in 1994. He received the B.S. degree in industrial engineering from the Wuhan University of Science and Technology, in 2016, where he is currently pursuing the Ph.D. degree in mechanical engineering. His research interests include assembly line balancing, assembly flow shop scheduling and optimization algorithms.

Qiuhua Tang was born in Lichuan, Hubei, China, in 1970. She received the B.S. degree in process and equipment of machinery manufacturing from Northeastern University, in 1992, and the master's degree in mechanical design and theory and the Ph.D. degree from the Wuhan University of Science and Technology. Since 1992, she has been working with the Wuhan University of Science and Technology, and was promoted to Professor, in 2009. Her research interests include production process planning and scheduling, manufacturing process monitoring and control, and modern optimization methods and algorithms. She has published more than 100 papers in academic journals and conferences. She has undertaken three general research projects supported by the National Natural Science Foundation of China (NSFC), and won the 2016 honorable mention in mechanical engineering discipline of NSFC, in December 2017. She also received awards, like Wuhan Excellent Paper Award and Excellent Paper of Chinese Mechanical Engineering Society.